목차

1. 정렬 / 이분 탐색 관련 1
1.1 정렬 비교 함수 구현 [PY] . . . 1
1.2 좌표 압축 [PY] . . . 2
1.3 이분 탐색과 선택 알고리즘 [PY] . . . 2

2. 그래프/트리 (Easy) 2
2.1 인접 리스트, DFS, BFS [PY] . . . 2
2.2 위상 정렬 [PY] . . . 2
2.3 최단 거리 - Floyd Warshall [PY/C++] . . . 3
2.4 최단 거리 - Dijkstra [PY/C++] . . . 3
2.5 최단 거리 - Bellman Ford [PY/C++] . . . 4
2.6 유니온 파인드 + 최소 신장 트리(Kruskal) [PY/C++] . . . 4

3. 자료구조 5
3.1 세그먼트 트리 [PY/C++] . . . 5
3.2 세그먼트 트리 + 레이지 프로퍼게이션 [C++] . . . 5
3.3 Convex Hull Trick [C++] . . . 6
3.4 퍼시스턴트 세그먼트 트리 [C++] . . . 6

4. 그래프/트리 (Hard) 7
4.1 SCC - Kosaraju [PY/C++] . . . 7
4.2 BCC - Tarjan [C++] . . . 7
4.3 최대 유량 - Dinic [C++] . . . 8
4.4 MCMF [C++] . . . 8
4.5 이분 매칭 - Hopcroft Karp [C++] . . . 9
4.6 최소 공통 조상(LCA) [PY/C++] . . . 9
4.7 Heavy Light Decomposition [C++] . . . 10

5. 수학 11
5.1 나눗셈, 최대공약수, 최소공배수 [PY] . . . 11
5.2 빠른 거듭제곱 [PY] . . . 11
5.3 소수 판별, 소인수분해 [PY] . . . 11
5.4 에라토스테네스의 체, 소인수분해 [PY] . . . 11
5.5 선형 시간 체, 곱셈적 함수 전처리 [C++] . . . 11
5.6 확장 유클리드 알고리즘 [PY] . . . 11
5.7 중국인의 나머지 정리 [PY/C++] . . . 12
5.8 이항 계수를 소수로 나눈 나머지 [PY] . . . 12
5.9 빠른 소수 판별, 소인수분해 - Miller Rabin, Pollard Rho [C++] . . . 12
5.10 가우스 소거법 - RREF, 랭크, 행렬식, 역행렬 [PY/C++] . . . 13
5.11 다항식 곱셈(FFT) [C++] . . . 14

6. 문자열 15
6.1 문자열 해싱 [C++] . . . 15
6.2 문자열 매칭 - KMP [PY/C++] . . . 15
6.3 가장 긴 팰린드롬 부분 문자열 - Manacher [PY/C++] . . . 16
6.4 문자열 매칭 - Z [PY/C++] . . . 16
6.5 접미사 배열 [C++] . . . 16

7. 계산 기하 17
7.1 2차원 계산 기하 템플릿 + CCW [PY/C++] . . . 17
7.2 360도 각도 정렬 [PY/C++] . . . 17
7.3 다각형 넓이 [PY] . . . 17
7.4 선분 교차 판정 [PY/C++] . . . 18
7.5 다각형 내부 판별 [PY/C++] . . . 18
7.6 볼록 껍질 - Monotone Chain [C++] . . . 18
7.7 가장 먼 두 점 - Rotating Calipers [C++] . . . 19
7.8 볼록 다각형 내부 판별 [C++] . . . 19

8. 기타 19
8.1 가장 긴 증가하는 부분 수열(LIS) [PY/C++] . . . 19
8.2 이분 탐색 [PY] . . . 19
8.3 삼분 탐색 [PY] . . . 20
8.4 Python 랜덤, 순서 통계, 비트마스킹 [PY] . . . 20
8.5 빠른 입력(Fast Input from STDIN) [PY/C++] . . . 20
8.6 구간별 약수 최대 개수, 최대 소수 . . . 21
8.7 카탈란 수, 심슨 적분, 그런디 정리, 픽의 정리, 페르마 포인트, 오일러 정리 . . . 21
8.8 경우의 수 - 포함 배제, 스털링 수, 벨 수 . . . 22
8.9 삼각형의 오심 - 외심, 내심, 무게중심, 수심, 방심 . . . 22
8.10 미적분, 뉴턴 랩슨법 . . . 23
8.11 문제 풀이 체크리스트 . . . 23

9. Python 추가 참조 23
9.1 Python 빠른 참조 [PY] . . . 23
9.2 문법과 성능 주의사항 [PY] . . . 23
9.3 곱셈표의 k번째 수 [PY] . . . 23
9.4 편집 거리와 DP 점검 [PY] . . . 24
9.5 수선의 발·거리·대칭점·교점 [PY] . . . 24
9.6 제출 전 추가 점검 [PY] . . . 24

0.1 C++17 공통 정의

```
#pragma once
#include <bits/stdc++.h>
// GNU C++17. 인덱스는 0부터, 기본 구간은 [l,r).
// 각 헤더를 포함하고 tn 네임스페이스의 함수를 사용한다.
namespace tn {
using namespace std;
using ll = long long;
using i128 = __int128_t;
using u64 = uint64_t;
using u128 = __uint128_t;
constexpr ll INF = numeric_limits<ll>::max() / 4;
// 거리·유량·비용의 중간 결과도 (-INF,INF) 안이어야 한다.
}
```

1. 정렬 / 이분 탐색 관련

1.1 정렬 비교 함수 구현 [PY]

```
# 안정 정렬 O(N log N). 같은 키의 원래 순서를 유지한다.
```

```
from functools import cmp_to_key

def sort_points(points):
    # x 오름차순, 같으면 y 오름차순.
    return sorted(points, key=lambda p: (p[0], p[1]))

def sort_with_comparator(values, compare):
    # 비교 함수는 a<b, a=b, a>b에 각각 음수·0·양수를 반환한다.
    return sorted(values, key=cmp_to_key(compare))
```

1.2 좌표 압축 [PY]

```
# 중복을 제거하고 정렬한 값에 0부터 번호를 붙인다.
def coordinate_compress(values):
    # (0부터 시작하는 압축 좌표, 복원용 정렬 고유 값). O(N log N), 공간 O(N).
    unique = sorted(set(values))
    rank = {value: i for i, value in enumerate(unique)}
    return [rank[value] for value in values], unique
```

1.3 이분 탐색과 선택 알고리즘 [PY]

```
# 정렬 배열의 경계 탐색 O(log N). 리스트 삽입은 O(N).
from bisect import bisect_left, bisect_right

def equal_range(values, x):
    # 정렬 배열에서 x가 나타나는 구간 [왼쪽, 오른쪽)을 구한다.
    return bisect_left(values, x), bisect_right(values, x)

def contains(values, x):
    i = bisect_left(values, x)
    return i < len(values) and values[i] == x

def nth_element(values, k):
    # 배열을 직접 바꾸어 k번째 값을 선택한다. 기대 O(N), 최악 O(N^2).
    from random import randrange
    if not 0 <= k < len(values):
        raise IndexError(k)
    l, r = 0, len(values)
    while r - l > 1:
        pivot = values[randrange(l, r)]
        a = i = l
        b = r
        while i < b:
            if values[i] < pivot:
                values[a], values[i] = values[i], values[a]
                a += 1
                i += 1
            elif values[i] > pivot:
                b -= 1
                values[b], values[i] = values[i], values[b]
            else:
                i += 1
        if k < a:
            r = a
        elif k >= b:
            l = b
        else:
            break
    return values[k]
```

2. 그래프/트리 (Easy)

2.1 인접 리스트, DFS, BFS [PY]

```
# 인접 리스트 순회 O(V+E). 정점 번호는 0부터 시작한다.
from collections import deque

from heapq import heappop, heappush

from math import inf

def bfs(graph, start):
    # 시작점에서의 간선 수 거리. 도달 불가는 -1.
    distance = [-1] * len(graph)
    distance[start] = 0
    queue = deque([start])
    while queue:
        v = queue.popleft()
        for u in graph[v]:
            if distance[u] == -1:
                distance[u] = distance[v] + 1
                queue.append(u)
    return distance

def dfs(graph, start):
    # 재귀 DFS와 같은 인접 리스트 순서로 방문. 재귀 깊이 제한 없음.
    seen = [False] * len(graph)
    seen[start] = True
    order = [start]
    stack = [iter(graph[start])]
    while stack:
        u = next(stack[-1], None)
        if u is None:
            stack.pop()
        elif not seen[u]:
            seen[u] = True
            order.append(u)
            stack.append(iter(graph[u]))
    return order
```

2.2 위상 정렬 [PY]

```
# 진입 차수가 0인 정점부터 꺼내 위상 순서를 구한다. O(V+E).
from collections import deque

from heapq import heappop, heappush

from math import inf

def topological_sort(graph):
    # DAG의 위상 순서. 사이클이 있으면 None. 빈 그래프는 [].
```

```
    indegree = [0] * len(graph)
    for neighbors in graph:
        for u in neighbors:
            indegree[u] += 1
    queue = deque(v for v, degree in enumerate(indegree) if degree == 0)
    order = []
    while queue:
        v = queue.popleft()
        order.append(v)
        for u in graph[v]:
            indegree[u] -= 1
            if indegree[u] == 0:
                queue.append(u)
    return order if len(order) == len(graph) else None
```

2.3 최단 거리 - Floyd Warshall [PY/C++]

```
# 모든 쌍 최단 거리 O(V^3). d[v][v]<0이면 음수 사이클이 있다.
# 음수 사이클을 거치는 경로의 거리는 유효하지 않다.
from math import inf

def floyd_warshall(n, edges):
    d = [[inf] * n for _ in range(n)]
    for v in range(n):
        d[v][v] = 0
    for u, v, w in edges:
        d[u][v] = min(d[u][v], w)
    for k in range(n):
        for i in range(n):
            if d[i][k] == inf:
                continue
            for j in range(n):
                d[i][j] = min(d[i][j], d[i][k] + d[k][j])
    return d
```

```
#pragma once
#include "../common.hpp"
// 모든 쌍 최단 거리 O(V^3), 공간 O(V^2).
namespace tn {
vector<vector<ll>> floyd_warshall(int n,const vector<tuple<int,int,ll>>& edge
    | s) {
    vector<vector<ll>> d(n,vector<ll>(n,INF));
    for (int i=0;i<n;i++) d[i][i]=0;
    for (auto [u,v,w]:edges) d[u][v]=min(d[u][v],w);
    for (int k=0;k<n;k++) for (int i=0;i<n;i++) if (d[i][k]!=INF)
        for (int j=0;j<n;j++) if (d[k][j]!=INF)
            d[i][j]=min(d[i][j],d[i][k]+d[k][j]);
    return d; // d[v][v]<0이면 음수 사이클. 이를 거치는 거리는 유효하지 않다.
}
}
```

2.4 최단 거리 - Dijkstra [PY/C++]

```
# 음이 아닌 가중치의 최단 거리. 최소 힙으로 다음 정점을 고른다.
from collections import deque

from heapq import heappop, heappush

from math import inf

def dijkstra(graph, start):
    # 인접 항목은 (정점, 가중치). (거리, 부모) 반환; 도달 불가는 (inf,-1).
    # 음수 가중치는 허용하지 않는다.
    if any(weight < 0 for edges in graph for _, weight in edges):
        raise ValueError('Dijkstra requires nonnegative weights')
    distance = [inf] * len(graph)
    parent = [-1] * len(graph)
    distance[start] = 0
    queue = [(0, start)]
    while queue:
        cost, v = heappop(queue)
        if cost != distance[v]:
            continue
        for u, weight in graph[v]:
            candidate = cost + weight
            if candidate < distance[u]:
                distance[u] = candidate
                parent[u] = v
                heappush(queue, (candidate, u))
    return distance, parent

def restore_path(parent, start, target):
    # dijkstra의 부모 배열에서 경로 복원. 도달 불가면 []. O(V).
    path = []
    while target != -1:
        path.append(target)
        if target == start:
            return path[::-1]
        target = parent[target]
    return []
```

```
#pragma once
#include "../common.hpp"
// 음이 아닌 가중치의 최단 거리. 최소 힙으로 다음 정점을 고른다.
namespace tn {
pair<vector<ll>,vector<int>> dijkstra(
    const vector<vector<pair<int,ll>>>& g, int s) {
    for (auto& row:g) for (auto [v,w]:row)
        if (w<0) throw invalid_argument("negative weight");
    vector<ll> d(g.size(),INF); vector<int> p(g.size(),-1);
    priority_queue<pair<ll,int>,vector<pair<ll,int>>,greater<pair<ll,int>>> q
    | ;
    d[s]=0; q.push({0,s});
    while (!q.empty()) {
        auto [cost,v]=q.top(); q.pop(); if (cost!=d[v]) continue;
        for (auto [u,w]:g[v]) if (cost+w<d[u]) {
            d[u]=cost+w; p[u]=v; q.push({d[u],u});
```

```
        }
    }
    return {d,p}; // 도달 불가는 거리 INF, 부모 -1.
}
vector<int> restore_path(const vector<int>& p,int s,int t) {
    vector<int> path;
    for (;t!=-1;t=p[t]) {
        path.push_back(t);
        if (t==s) { reverse(path.begin(),path.end()); return path; }
    }
    return {};
}
}
```

2.5 최단 거리 - Bellman Ford [PY/C++]

```
# 간선 (u,v,w)를 반복 완화한다. O(VE). 도달 가능한 음수 사이클은 None.
from math import inf

def bellman_ford(n, edges, start):
    edges = list(edges)
    d = [inf] * n
    d[start] = 0
    for step in range(n):
        changed = False
        for u, v, w in edges:
            if d[u] != inf and d[v] > d[u] + w:
                d[v] = d[u] + w
                changed = True
        if not changed:
            return d
    return None
#pragma once
#include "../common.hpp"
// 간선을 반복 완화하는 최단 거리 O(VE). 음수 사이클이면 nullopt.
namespace tn {
optional<vector<ll>> bellman_ford(int n,const vector<tuple<int,int,ll>>& e,in
    | t s) {
    vector<ll> d(n,INF); d[s]=0;
    for (int step=0;step<n;step++) {
        bool changed=false;
        for (auto [u,v,w]:e) if (d[u]!=INF && d[v]>d[u]+w)
            d[v]=d[u]+w, changed=true;
        if (!changed) return d;
    }
    return nullopt; // 도달 가능한 음수 사이클.
}
}
```

2.6 유니온 파인드 + 최소 신장 트리(Kruskal) [PY/C++]

```
# 간선을 가중치순으로 골라 최소 신장 포리스트를 만든다. O(V+E log(E+1)).
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.size = [1] * n
        self.components = n

    def find(self, v):
        while self.parent[v] != v:
            self.parent[v] = self.parent[self.parent[v]]
            v = self.parent[v]
        return v

    def union(self, u, v):
        u, v = self.find(u), self.find(v)
        if u == v:
            return False
        if self.size[u] < self.size[v]:
            u, v = v, u
        self.parent[v] = u
        self.size[u] += self.size[v]
        self.components -= 1
        return True

def kruskal(n, edges):
    # 간선은 (가중치,u,v). (비용, 선택 간선) 반환; 연결 그래프면 간선 수 n-1.
    dsu = DSU(n)
    total = 0
    selected = []
    for weight, u, v in sorted(edges):
        if dsu.union(u, v):
            total += weight
            selected.append((weight, u, v))
    return total, selected
#pragma once
#include "../common.hpp"
// 가중치순으로 간선을 선택하는 최소 신장 포리스트. O(V+E log E).
namespace tn {
struct DSU {
    vector<int> parent,size; int components;
    explicit DSU(int n):parent(n),size(n,1),components(n) {
        iota(parent.begin(),parent.end(),0);
    }
    int find(int v) {
        while (parent[v]!=v) v=parent[v]=parent[parent[v]];
        return v;
    }
    bool unite(int u,int v) {
        u=find(u); v=find(v); if (u==v) return false;
        if (size[u]<size[v]) swap(u,v);
        parent[v]=u; size[u]+=size[v]; --components; return true;
    }
};
pair<ll,vector<tuple<ll,int,int>>> kruskal(int n,vector<tuple<ll,int,int>> ed
```

```
    | ges) {
    sort(edges.begin(),edges.end()); DSU d(n); ll cost=0;
    vector<tuple<ll,int,int>> chosen;
    for (auto [w,u,v]:edges) if (d.unite(u,v)) cost+=w,chosen.push_back({w,u,
    | v});
    return {cost,chosen}; // 비연결이면 포리스트. 간선은 (가중치,u,v).
}
}
```

3. 자료구조

3.1 세그먼트 트리 [PY/C++]

```
# 점 대입과 구간 [l,r) 합 O(log N). 초기화 O(N).

class SegmentTree:
    def __init__(self, values):
        self.n = len(values)
        self.size = 1 << max(0, (self.n - 1).bit_length())
        self.tree = [0] * (2 * self.size)
        self.tree[self.size:self.size + self.n] = values
        for i in range(self.size - 1, 0, -1):
            self.tree[i] = self.tree[i * 2] + self.tree[i * 2 + 1]

    def set(self, i, value):
        if not 0 <= i < self.n:
            raise IndexError(i)
        i += self.size
        self.tree[i] = value
        while i > 1:
            i //= 2
            self.tree[i] = self.tree[i * 2] + self.tree[i * 2 + 1]

    def sum(self, l, r):
        if not 0 <= l <= r <= self.n:
            raise IndexError((l, r))
        answer = 0
        l += self.size
        r += self.size
        while l < r:
            if l & 1:
                answer += self.tree[l]
                l += 1
            if r & 1:
                r -= 1
                answer += self.tree[r]
            l //= 2
            r //= 2
        return answer
```

```
#pragma once
#include "../common.hpp"
// 점 대입과 구간 [l,r) 합 O(log N). 초기화 O(N).
namespace tn {
struct SegmentTree {
    int n,size=1; vector<ll> tree;
    explicit SegmentTree(const vector<ll>& a):n(a.size()) {
        while (size<n) size*=2;
        tree.assign(2*size,0); copy(a.begin(),a.end(),tree.begin()+size);
        for (int i=size-1;i;i--) tree[i]=tree[i*2]+tree[i*2+1];
    }
    void set(int i,ll value) {
        assert(0<=i && i<n); tree[i+=size]=value;
        while (i/=2) tree[i]=tree[i*2]+tree[i*2+1];
    }
    ll sum(int l,int r) const {
        assert(0<=l && l<=r && r<=n); ll result=0;
        for (l+=size,r+=size;l<r;l/=2,r/=2) {
            if (l&1) result+=tree[l++];
            if (r&1) result+=tree[--r];
        }
        return result;
    }
};
}
```

3.2 세그먼트 트리 + 레이지 프로퍼게이션 [C++]

```
#pragma once
#include "../common.hpp"
// 구간 [l,r) 덧셈과 합 O(log N). 갱신을 자식에 지연 전파한다.
namespace tn {
struct LazySegmentTree {
    int n; vector<ll> tree,lazy;
    explicit LazySegmentTree(int n):n(n),tree(4*max(1,n)),lazy(tree.size()) {
    | }
    void apply(int v,int l,int r,ll x) { tree[v]+=(r-l)*x; lazy[v]+=x; }
    void push(int v,int l,int r) {
        int m=(l+r)/2;
        apply(v*2,l,m,lazy[v]); apply(v*2+1,m,r,lazy[v]); lazy[v]=0;
    }
    ll query(int v,int l,int r,int a,int b,optional<ll> x) {
        if (b<=l || r<=a) return 0;
        if (a<=l && r<=b) { if (x) apply(v,l,r,*x); return tree[v]; }
        push(v,l,r); int m=(l+r)/2;
        ll answer=query(v*2,l,m,a,b,x)+query(v*2+1,m,r,a,b,x);
        tree[v]=tree[v*2]+tree[v*2+1]; return answer;
    }
    void add(int l,int r,ll x) {
        assert(0<=l && l<=r && r<=n); if (l<r) query(1,0,n,l,r,x);
    }
    ll sum(int l,int r) {
        assert(0<=l && l<=r && r<=n); return l<r?query(1,0,n,l,r,nullopt):0;
    }
};
}
```

3.3 Convex Hull Trick [C++]

```
#pragma once
#include "../common.hpp"
// 직선 최댓값 질의. 단조형과 Li Chao 트리를 제공한다.
// 기울기·절편·x의 절댓값은 10^9 이하. 최솟값은 부호를 뒤집는다.
namespace tn {
struct Line {
    ll a,b; int id=-1;
    ll at(ll x) const { return a*x+b; }
};
struct MonotoneCHT {
    deque<Line> q; optional<ll> last_a,last_x;
    void add(ll a,ll b,int id=-1) { // 기울기 비감소 순서, 분할 상환 O(1).
        assert(!last_a || a>=*last_a); last_a=a;
        if (!q.empty() && q.back().a==a) {
            if (q.back().b>=b) return; q.pop_back();
        }
        while (q.size()>1) {
            auto x=q[q.size()-2],y=q.back();
            if ((i128(x.b)-y.b)*(a-y.a)<(i128(y.b)-b)*(y.a-x.a)) break;
            q.pop_back();
        }
        q.push_back({a,b,id});
    }
    pair<ll,int> query(ll x) { // 질의 x 비감소 순서, 분할 상환 O(1).
        assert(!q.empty() && (!last_x || x>=*last_x)); last_x=x;
        while (q.size()>1 && q[0].at(x)<=q[1].at(x)) q.pop_front();
        return {q.front().at(x),q.front().id};
    }
};
struct LiChao { // 임의 순서, 고정 정수 구간 [lo,hi], O(log 범위).
    struct Node { Line line; int l=-1,r=-1; };
    ll lo,hi; vector<Node> t;
    LiChao(ll lo,ll hi):lo(lo),hi(hi) { assert(lo<=hi); }
    void add(ll a,ll b) {
        Line line{a,b}; if (t.empty()) { t.push_back({line}); return; }
        int v=0; ll l=lo,r=hi;
        while (true) {
            ll m=l+(r-l)/2;
            bool left=line.at(l)>t[v].line.at(l),mid=line.at(m)>t[v].line.at(
    | m);
            if (mid) swap(line,t[v].line);
            if (l==r) return;
            bool go_left=left!=mid; int child=go_left?t[v].l:t[v].r;
            if (child==-1) {
                child=t.size(); if (go_left) t[v].l=child; else t[v].r=child;
                t.push_back({line}); return;
            }
            v=child; if (go_left) r=m; else l=m+1;
        }
    }
    ll query(ll x) const {
        assert(!t.empty() && lo<=x && x<=hi);
        ll result=-INF,l=lo,r=hi; int v=0;
        while (v!=-1) {
            result=max(result,t[v].line.at(x)); if (l==r) break;
            ll m=l+(r-l)/2;
            if (x<=m) v=t[v].l,r=m; else v=t[v].r,l=m+1;
        }
        return result;
    }
};
}
```

3.4 퍼시스턴트 세그먼트 트리 [C++]

```
#pragma once
#include "../common.hpp"
// 빈도 트리의 버전 보존. 삽입·k번째 질의 O(log N), 공간 O(Q log N).
namespace tn {
struct PersistentSegmentTree {
    struct Node { int l=0,r=0,count=0; };
    int n; vector<Node> nodes{{}}; vector<int> roots{0};
    explicit PersistentSegmentTree(int n):n(n) { assert(n>0); }
    int update(int old,int l,int r,int x) {
        int v=nodes.size(); nodes.push_back(nodes[old]); ++nodes[v].count;
        if (r-l>1) {
            int m=(l+r)/2;
            if (x<m) { int child=update(nodes[old].l,l,m,x); nodes[v].l=child
    | ; }
            else { int child=update(nodes[old].r,m,r,x); nodes[v].r=child; }
        }
        return v;
    }
    int insert(int version,int x) {
        assert(0<=x && x<n); int root=update(roots.at(version),0,n,x);
        roots.push_back(root); return int(roots.size())-1;
    }
    int kth(int before,int after,int k) const {
        int a=roots.at(before),b=roots.at(after),l=0,r=n;
        assert(1<=k && k<=nodes[b].count-nodes[a].count);
        // 버전 간 빈도 차는 음수가 아니어야 하며 k는 1부터 시작한다.
        while (r-l>1) {
            int m=(l+r)/2, count=nodes[nodes[b].l].count-nodes[nodes[a].l].co
    | unt;
            if (k<=count) a=nodes[a].l,b=nodes[b].l,r=m;
            else k-=count,a=nodes[a].r,b=nodes[b].r,l=m;
        }
        return l;
    }
};
}
```

4. 그래프/트리 (Hard)

4.1 SCC - Kosaraju [PY/C++]

```
# 반복형 코사라주로 강한 연결 요소를 구한다. O(V+E).
# 요소 번호는 축약 그래프의 위상 순서다.

def strongly_connected_components(graph):
    n = len(graph)
    reverse = [[] for _ in graph]
    for v in range(n):
        for u in graph[v]:
            reverse[u].append(v)
    seen = [False] * n
    order = []
    for start in range(n):
        if seen[start]:
            continue
        seen[start] = True
        stack = [(start, iter(graph[start]))]
        while stack:
            v, children = stack[-1]
            u = next(children, None)
            if u is None:
                order.append(v)
                stack.pop()
            elif not seen[u]:
                seen[u] = True
                stack.append((u, iter(graph[u])))
    component = [-1] * n
    groups = []
    for start in reversed(order):
        if component[start] != -1:
            continue
        component[start] = len(groups)
        group, stack = [], [start]
        while stack:
            v = stack.pop()
            group.append(v)
            for u in reverse[v]:
                if component[u] == -1:
                    component[u] = component[start]
                    stack.append(u)
        groups.append(group)
    return component, groups
#pragma once
#include "../common.hpp"
// 반복형 코사라주의 강한 연결 요소 분해 O(V+E).
namespace tn {
pair<vector<int>,vector<vector<int>>> strongly_connected_components(
    const vector<vector<int>>& g) {
    int n=g.size(); vector<vector<int>> rev(n),groups;
    for (int v=0;v<n;v++) for (int u:g[v]) rev[u].push_back(v);
    vector<int> seen(n),order,comp(n,-1),pos(n);
    for (int s=0;s<n;s++) if (!seen[s]) {
        vector<int> st{s}; seen[s]=1;
        while (!st.empty()) {
            int v=st.back();
            if (pos[v]==int(g[v].size())) { order.push_back(v); st.pop_back()
    | ; }
            else { int u=g[v][pos[v]++]; if (!seen[u]) seen[u]=1,st.push_back
    | (u); }
        }
    }
    reverse(order.begin(),order.end());
    for (int s:order) if (comp[s]==-1) {
        int id=groups.size(); groups.push_back({}); comp[s]=id; vector<int> s
    | t{s};
        while (!st.empty()) {
            int v=st.back(); st.pop_back(); groups.back().push_back(v);
            for (int u:rev[v]) if (comp[u]==-1) comp[u]=id,st.push_back(u);
        }
    }
    return {comp,groups}; // 요소 번호는 축약 그래프의 위상 순서.
}
}
```

4.2 BCC - Tarjan [C++]

```
#pragma once
#include "../common.hpp"
// 반복형 타잔으로 단절점·단절선·이중 연결 요소를 구한다. O(V+E).
// 무향 다중 그래프, 자기 루프 금지. 단절점은 여러 묶음에 속한다.
namespace tn {
struct BCCResult { vector<int> cuts,bridges; vector<vector<int>> groups; };
BCCResult biconnected_components(int n,const vector<pair<int,int>>& edges) {
    vector<vector<pair<int,int>>> g(n);
    for (int i=0;i<int(edges.size());i++) {
        auto [u,v]=edges[i]; if (u==v) throw invalid_argument("self loop");
        g[u].push_back({v,i}); g[v].push_back({u,i});
    }
    vector<int> tin(n,-1),low(n),par(n,-1),pe(n,-1),pos(n),children(n),cut(n)
    | ;
    vector<int> edge_stack,mark(n,-1); int timer=0; BCCResult out;
    for (int root=0;root<n;root++) if (tin[root]==-1) {
        vector<int> st{root}; tin[root]=low[root]=timer++;
        while (!st.empty()) {
            int v=st.back();
            if (pos[v]<int(g[v].size())) {
                auto [u,e]=g[v][pos[v]++]; if (e==pe[v]) continue;
                if (tin[u]==-1) {
                    par[u]=v; pe[u]=e; children[v]++; edge_stack.push_back(e)
    | ;
                    tin[u]=low[u]=timer++; st.push_back(u);
```

```
            } else if (tin[u]<tin[v]) {
                low[v]=min(low[v],tin[u]); edge_stack.push_back(e);
            }
            continue;
        }
        st.pop_back(); int p=par[v];
        if (p==-1) {
            cut[v]=children[v]>1;
            if (g[v].empty()) out.groups.push_back({v});
        } else {
            low[p]=min(low[p],low[v]);
            if (low[v]>tin[p]) out.bridges.push_back(pe[v]);
            if (low[v]>=tin[p]) {
                if (par[p]!=-1) cut[p]=1;
                int id=out.groups.size(); out.groups.push_back({});
                while (true) {
                    int e=edge_stack.back(); edge_stack.pop_back();
                    for (int x:{edges[e].first,edges[e].second}) if (mark
| [x]!=id)
                        mark[x]=id,out.groups.back().push_back(x);
                    if (e==pe[v]) break;
                }
            }
        }
    }
    for (int v=0;v<n;v++) if (cut[v]) out.cuts.push_back(v);
    return out; // 단절선은 입력 간선 번호, 고립점도 별도 묶음.
}
}
```

4.3 최대 유량 - Dinic [C++]

```
#pragma once
#include "../common.hpp"
// 레벨 그래프를 이용한 최대 유량 O(V^2 E).
namespace tn {
struct Dinic {
    vector<vector<int>> g; vector<int> to; vector<ll> cap;
    explicit Dinic(int n):g(n) {}
    int add_edge(int u,int v,ll c,ll reverse=0) {
        assert(c>=0 && reverse>=0); int e=to.size();
        g[u].push_back(e); g[v].push_back(e^1);
        to.push_back(v); to.push_back(u); cap.push_back(c); cap.push_back(rev
| erse);
        return e;
    }
    vector<int> levels(int s) const {
        vector<int> d(g.size(),-1); queue<int> q; d[s]=0; q.push(s);
        while (!q.empty()) {
            int v=q.front(); q.pop();
            for (int e:g[v]) if (cap[e]>0 && d[to[e]]==-1)
                d[to[e]]=d[v]+1,q.push(to[e]);
        }
        return d;
    }
    ll max_flow(int s,int t,ll limit=INF) { // 잔여 용량을 갱신하고 추가 유량을
| 반환한다.
        assert(s!=t); ll flow=0;
        while (flow<limit) {
            auto level=levels(s); if (level[t]==-1) break;
            vector<int> pos(g.size());
            while (flow<limit) {
                vector<int> vertices{s},path;
                while (!vertices.empty() && vertices.back()!=t) {
                    int v=vertices.back();
                    while (pos[v]<int(g[v].size())) {
                        int e=g[v][pos[v]];
                        if (cap[e]>0 && level[to[e]]==level[v]+1) break;
                        ++pos[v];
                    }
                    if (pos[v]==int(g[v].size())) {
                        level[v]=-1; vertices.pop_back();
                        if (!path.empty()) path.pop_back();
                    } else { int e=g[v][pos[v]]; path.push_back(e); vertices.
| push_back(to[e]); }
                }
                if (vertices.empty()) break;
                ll amount=limit-flow; for (int e:path) amount=min(amount,cap[
| e]);
                for (int e:path) cap[e]-=amount,cap[e^1]+=amount;
                flow+=amount;
            }
        }
        return flow;
    }
    pair<vector<int>,vector<int>> min_cut(int s) const { // 유량 제한 없이 최대
|  유량을 구한 뒤 호출한다.
        auto d=levels(s); vector<int> a,b;
        for (int v=0;v<int(g.size());v++) (d[v]==-1?b:a).push_back(v);
        return {a,b};
    }
};
}
```

4.4 MCMF [C++]

```
#pragma once
#include "../common.hpp"
// 최단 증가 경로로 최소 비용 유량을 구한다. O(A VE), A는 증가 횟수.
// 도달 가능한 음수 사이클 금지. 누적 비용은 (-INF,INF) 범위.
namespace tn {
struct MinCostFlow {
    struct Edge { int u,v; ll cap,cost; };
```

```
    int n; vector<Edge> edges;
    explicit MinCostFlow(int n):n(n) {}
    void add_edge(int u,int v,ll cap,ll cost) {
        assert(cap>=0); edges.push_back({u,v,cap,cost}); edges.push_back({v,u
    | ,0,-cost});
    }
    pair<ll,ll> run(int s,int t,ll limit=INF) { // 추가 (유량, 비용)을 반환한다
    | .
        assert(s!=t); ll flow=0,cost=0;
        while (flow<limit) {
            vector<ll> d(n,INF); vector<int> p(n,-1); d[s]=0;
            for (int step=0;step<n;step++) {
                bool changed=false;
                for (int i=0;i<int(edges.size());i++) {
                    auto e=edges[i];
                    if (e.cap>0 && d[e.u]!=INF && d[e.v]>d[e.u]+e.cost)
                        d[e.v]=d[e.u]+e.cost,p[e.v]=i,changed=true;
                }
                if (!changed) break;
                if (step==n-1) throw invalid_argument("negative cost cycle");
            }
            if (d[t]==INF) break;
            ll amount=limit-flow;
            for (int v=t;v!=s;v=edges[p[v]].u) amount=min(amount,edges[p[v]].
    | cap);
            for (int v=t;v!=s;v=edges[p[v]].u) {
                edges[p[v]].cap-=amount; edges[p[v]^1].cap+=amount;
            }
            flow+=amount; cost+=amount*d[t];
        }
        return {flow,cost};
    }
};
}
```

4.5 이분 매칭 - Hopcroft Karp [C++]

```
#pragma once
#include "../common.hpp"
// 층별 증가 경로를 찾는 최대 이분 매칭 O((V+E)sqrt(V)).
namespace tn {
struct MatchingResult { vector<int> left,right,cover_left,cover_right; };
MatchingResult hopcroft_karp(const vector<vector<int>>& g,int m) {
    int n=g.size(); vector<int> le(n,-1),ri(m,-1);
    while (true) {
        vector<int> d(n,-1),pos(n); queue<int> q; int shortest=n+1;
        for (int v=0;v<n;v++) if (le[v]==-1) d[v]=0,q.push(v);
        while (!q.empty()) {
            int v=q.front(); q.pop(); if (d[v]>=shortest) continue;
            for (int u:g[v]) {
                int w=ri[u];
                if (w==-1) shortest=d[v]+1;
                else if (d[w]==-1) d[w]=d[v]+1,q.push(w);
            }
        }
        if (shortest==n+1) break;
        for (int s=0;s<n;s++) if (le[s]==-1) {
            vector<int> st{s},links;
            while (!st.empty()) {
                int v=st.back();
                if (pos[v]==int(g[v].size())) {
                    d[v]=-1; st.pop_back(); if (!links.empty()) links.pop_bac
    | k(); continue;
                }
                int u=g[v][pos[v]++],w=ri[u];
                if (w==-1 && d[v]+1==shortest) {
                    links.push_back(u);
                    for (int i=0;i<int(st.size());i++) le[st[i]]=links[i],ri[
    | links[i]]=st[i];
                    break;
                }
                if (w!=-1 && d[w]==d[v]+1) st.push_back(w),links.push_back(u)
    | ;
            }
        }
    }
    vector<int> sl(n),sr(m),a,b; queue<int> q;
    for (int v=0;v<n;v++) if (le[v]==-1) sl[v]=1,q.push(v);
    while (!q.empty()) {
        int v=q.front(); q.pop();
        for (int u:g[v]) if (u!=le[v] && !sr[u]) {
            sr[u]=1; int w=ri[u]; if (w!=-1 && !sl[w]) sl[w]=1,q.push(w);
        }
    }
    for (int v=0;v<n;v++) if (!sl[v]) a.push_back(v);
    for (int u=0;u<m;u++) if (sr[u]) b.push_back(u);
    return {le,ri,a,b}; // 최소 정점 커버는 cover_left와 cover_right의 합집합.
}
}
```

4.6 최소 공통 조상(LCA) [PY/C++]

```
# 연결된 무향 트리의 최소 공통 조상. 전처리 O(N log N), 질의 O(log N).

class LCA:
    def __init__(self, graph, root=0):
        n = len(graph)
        self.depth = [-1] * n
        self.depth[root] = 0
        parent = [root] * n
        order = [root]
        for v in order:
            for u in graph[v]:
                if self.depth[u] == -1:
```

```
                    parent[u] = v
                    self.depth[u] = self.depth[v] + 1
                    order.append(u)
        if len(order) != n or sum(map(len, graph)) != 2 * (n - 1):
            raise ValueError('connected undirected tree required')
        self.up = [parent]
        for _ in range(1, n.bit_length()):
            prev = self.up[-1]
            self.up.append([prev[prev[v]] for v in range(n)])

    def query(self, u, v):
        if self.depth[u] < self.depth[v]:
            u, v = v, u
        diff = self.depth[u] - self.depth[v]
        for k in range(len(self.up)):
            if diff >> k & 1:
                u = self.up[k][u]
        if u == v:
            return u
        for level in reversed(self.up):
            if level[u] != level[v]:
                u, v = level[u], level[v]
        return self.up[0][u]
```

```
#pragma once
#include "../common.hpp"
// 연결된 무향 트리의 최소 공통 조상. 전처리 O(N log N), 질의 O(log N).
namespace tn {
struct LCA {
    vector<int> depth; vector<vector<int>> up;
    LCA(const vector<vector<int>>& g,int root=0):depth(g.size(),-1) {
        int n=g.size(),levels=1; while ((1LL<<levels)<=n) ++levels;
        up.assign(levels,vector<int>(n,root)); vector<int> order{root}; depth
    | [root]=0;
        for (int i=0;i<int(order.size());i++) {
            int v=order[i];
            for (int u:g[v]) if (depth[u]==-1)
                depth[u]=depth[v]+1,up[0][u]=v,order.push_back(u);
        }
        assert(int(order.size())==n);
        for (int k=1;k<levels;k++) for (int v=0;v<n;v++) up[k][v]=up[k-1][up[
    | k-1][v]];
    }
    int query(int u,int v) const { // 최소 공통 조상 질의 O(log N).
        if (depth[u]<depth[v]) swap(u,v);
        int diff=depth[u]-depth[v];
        for (int k=0;k<int(up.size());k++) if (diff>>k&1) u=up[k][u];
        if (u==v) return u;
        for (int k=int(up.size())-1;k>=0;k--) if (up[k][u]!=up[k][v])
            u=up[k][u],v=up[k][v];
        return up[0][u];
    }
};
}
```

4.7 Heavy Light Decomposition [C++]

```
#pragma once
#include "../common.hpp"
// 트리 경로를 O(log N)개 구간으로 분해한다. 초기화 O(N).
// 세그먼트 트리와 결합: 점 갱신 O(log N), 경로 합 O(log^2 N).
namespace tn {
struct HLD {
    vector<int> parent,depth,size,head,pos;
    HLD(const vector<vector<int>>& g,int root=0) {
        int n=g.size(); parent.assign(n,-1); depth.assign(n,0); size.assign(n
    | ,1);
        head.resize(n); pos.resize(n); vector<int> order{root},heavy(n,-1); p
    | arent[root]=root;
        for (int i=0;i<int(order.size());i++) {
            int v=order[i];
            for (int u:g[v]) if (parent[u]==-1)
                parent[u]=v,depth[u]=depth[v]+1,order.push_back(u);
        }
        assert(int(order.size())==n);
        for (int i=n-1;i>0;i--) {
            int v=order[i],p=parent[v]; size[p]+=size[v];
            if (heavy[p]==-1 || size[v]>size[heavy[p]]) heavy[p]=v;
        }
        vector<pair<int,int>> st{{root,root}}; int timer=0;
        while (!st.empty()) {
            auto [v,h]=st.back(); st.pop_back();
            for (;v!=-1;v=heavy[v]) {
                head[v]=h; pos[v]=timer++;
                for (int u:g[v]) if (parent[u]==v && u!=v && u!=heavy[v]) st.
    | push_back({u,u});
            }
        }
    }
    vector<pair<int,int>> path_ranges(int u,int v,bool edges=false) const {
        vector<pair<int,int>> ranges; // 구간 순서 미보장: 교환 가능한 연산만 사
    | 용한다.
        while (head[u]!=head[v]) {
            if (depth[head[u]]<depth[head[v]]) swap(u,v);
            ranges.push_back({pos[head[u]],pos[u]+1}); u=parent[head[u]];
        }
        if (depth[u]>depth[v]) swap(u,v);
        int l=pos[u]+edges,r=pos[v]+1; if (l<r) ranges.push_back({l,r});
        return ranges; // 간선 값은 더 깊은 정점 위치에 저장한다.
    }
    pair<int,int> subtree_range(int v) const { return {pos[v],pos[v]+size[v]}
    | ; }
};
}
```

5. 수학

5.1 나눗셈, 최대공약수, 최소공배수 [PY]

```
# 음수도 처리하는 정수 나눗셈·최대공약수·최소공배수. 나누는 수는 0이 아니어야 한다.
from math import gcd, lcm

def floor_div(a, b):
    return a // b

def ceil_div(a, b):
    return -(-a // b)
```

5.2 빠른 거듭제곱 [PY]

```
# 모듈러 거듭제곱 O(log 지수). 지수는 0 이상, 모듈러스는 1 이상.

def pow_mod(base, exponent, modulus):
    if exponent < 0 or modulus < 1:
        raise ValueError('exponent >= 0 and modulus >= 1 required')
    return pow(base, exponent, modulus)
```

5.3 소수 판별, 소인수분해 [PY]

```
# 제곱근까지 나누어 소인수분해한다. O(sqrt(N)).
from math import isqrt

def is_prime(n):
    return n >= 2 and all(n % p for p in range(2, isqrt(n) + 1))

def factorize(n):
    if n < 1:
        raise ValueError('positive integer required')
    result, p = [], 2
    while p * p <= n:
        count = 0
        while n % p == 0:
            n //= p
            count += 1
        if count:
            result.append((p, count))
        p += 1
    if n > 1:
        result.append((n, 1))
    return result
```

5.4 에라토스테네스의 체, 소인수분해 [PY]

```
# 최소 소인수 표를 만든다. O(N log log N), spf[1]=1.
# 표를 이용한 소인수분해는 O(log x).

def sieve(n):
    spf = list(range(n + 1))
    for p in range(2, int(n ** 0.5) + 1):
        if spf[p] == p:
            for x in range(p * p, n + 1, p):
                if spf[x] == x:
                    spf[x] = p
    return spf

def factorize(n, spf):
    if not 1 <= n < len(spf):
        raise ValueError('n outside sieve')
    result = []
    while n > 1:
        p, count = spf[n], 0
        while n % p == 0:
            n //= p
            count += 1
        result.append((p, count))
    return result
```

5.5 선형 시간 체, 곱셈적 함수 전처리 [C++]

```
#pragma once
#include "../common.hpp"
// N까지 소수·최소 소인수·오일러 함수·뫼비우스 함수·약수 개수·합을 구한다. O(N).
namespace tn {
struct LinearSieve {
    vector<int> primes,spf,phi,mu,tau; vector<ll> sigma;
    explicit LinearSieve(int n):spf(n+1),phi(n+1),mu(n+1),tau(n+1),sigma(n+1)
    |  {
        vector<int> exponent(n+1),power(n+1);
        if (n) spf[1]=phi[1]=mu[1]=tau[1]=sigma[1]=1;
        for (int i=2;i<=n;i++) {
            if (!spf[i]) {
                primes.push_back(i); spf[i]=power[i]=i; exponent[i]=1;
                phi[i]=i-1; mu[i]=-1; tau[i]=2; sigma[i]=i+1;
            }
            for (int p:primes) {
                if (ll(i)*p>n) break; int x=i*p; spf[x]=p;
                if (i%p==0) {
                    exponent[x]=exponent[i]+1; power[x]=power[i]*p;
                    phi[x]=phi[i]*p; mu[x]=0;
                    tau[x]=tau[i]/(exponent[i]+1)*(exponent[x]+1);
                    sigma[x]=sigma[i]+ll(power[x])*sigma[i/power[i]]; break;
                }
                exponent[x]=1; power[x]=p; phi[x]=phi[i]*(p-1); mu[x]=-mu[i];
                tau[x]=tau[i]*2; sigma[x]=sigma[i]*(p+1);
            }
        }
    }
};
}
```

5.6 확장 유클리드 알고리즘 [PY]

```
# ax+by=gcd(a,b)를 만족하는 (g,x,y)를 구한다. 음수·0도 처리한다.

def extended_gcd(a, b):
    old_r, r, old_x, x, old_y, y = a, b, 1, 0, 0, 1
```

```
    while r:
        q = old_r // r
        old_r, r = r, old_r - q * r
        old_x, x = x, old_x - q * x
        old_y, y = y, old_y - q * y
    sign = -1 if old_r < 0 else 1
    return sign * old_r, sign * old_x, sign * old_y
```

5.7 중국인의 나머지 정리 [PY/C++]

```
# 양의 모듈러스인 합동식을 병합한다. (나머지, 최소공배수), 불가능하면 None.
# 빈 입력은 (0,1). O(K log M).
from math import gcd

def crt(congruences):
    a, m = 0, 1
    for b, n in congruences:
        if n <= 0:
            raise ValueError('positive modulus required')
        g = gcd(m, n)
        if (b - a) % g:
            return None
        reduced = n // g
        t = ((b - a) // g * pow(m // g, -1, reduced)) % reduced
        a += m * t
        m *= reduced
        a %= m
    return a, m
```

```
#pragma once
#include "../common.hpp"
// 일반 중국인의 나머지 정리. 합동식 병합 O(K log M).
// 양의 모듈러스, 최종 최소공배수는 부호 있는 64비트 범위. 불가능하면 nullopt.
namespace tn {
tuple<i128,i128,i128> extended_gcd128(i128 a,i128 b) {
    i128 x=1,y=0,u=0,v=1;
    while (b) { i128 q=a/b; tie(a,b)=make_tuple(b,a-q*b);
        tie(x,u)=make_tuple(u,x-q*u); tie(y,v)=make_tuple(v,y-q*v); }
    return {a,x,y};
}
optional<pair<ll,ll>> crt(const vector<pair<ll,ll>>& equations) {
    i128 a=0,m=1;
    for (auto [bb,nn]:equations) {
        if (nn<=0) throw invalid_argument("positive modulus required");
        i128 b=bb,n=nn; auto [g,x,y]=extended_gcd128(m,n);
        if ((b-a)%g) return nullopt;
        i128 reduced=n/g,t=((b-a)/g*x)%reduced;
        if (t<0) t+=reduced;
        a+=m*t; m*=reduced;
        if (m>LLONG_MAX) throw overflow_error("CRT modulus overflow");
        a%=m; if (a<0) a+=m;
    }
    return pair<ll,ll>{ll(a),ll(m)};
}
}
```

5.8 이항 계수를 소수로 나눈 나머지 [PY]

```
# 소수 p에 대한 이항 계수. 전처리 O(p), 질의 O(log_p(n+1)).

class Lucas:
    def __init__(self, p):
        if p < 2:
            raise ValueError('prime p >= 2 required')
        self.p = p
        self.fac = [1] * p
        self.inv = [1] * p
        for i in range(1, p):
            self.fac[i] = self.fac[i - 1] * i % p
        self.inv[-1] = pow(self.fac[-1], p - 2, p)
        for i in range(p - 2, -1, -1):
            self.inv[i] = self.inv[i + 1] * (i + 1) % p

    def calc(self, n, r):
        if r < 0 or n < r:
            return 0
        result, p = 1, self.p
        while n or r:
            a, b = n % p, r % p
            if b > a:
                return 0
            result = result * self.fac[a] * self.inv[b] * self.inv[a - b] % p
            n //= p
            r //= p
        return result
```

5.9 빠른 소수 판별, 소인수분해 - Miller Rabin, Pollard Rho [C++]

```
#pragma once
#include "../common.hpp"
// uint64_t 전체 범위의 소수 판별·소인수분해.
// 밀러 라빈은 확정 판별, 폴라드 로는 기대 O(sqrt(최소 소인수)).
namespace tn {
u64 mul_mod(u64 a,u64 b,u64 m) { return u128(a)*b%m; }
u64 pow_mod64(u64 a,u64 b,u64 m) {
    u64 result=1%m;
    for (a%=m;b;b>>=1,a=mul_mod(a,a,m)) if (b&1) result=mul_mod(result,a,m);
    return result;
}
bool is_prime64(u64 n) {
    if (n<2) return false;
    for (u64 p:{2,3,5,7,11,13,17,19,23,29,31,37}) if (n%p==0) return n==p;
    u64 d=n-1; int s=0; while (!(d&1)) d>>=1,++s;
    for (u64 a:{2ULL,325ULL,9375ULL,28178ULL,450775ULL,9780504ULL,1795265022U
    | LL}) {
        if (a%n==0) continue; u64 x=pow_mod64(a,d,n);
        if (x==1 || x==n-1) continue;
```

```
        bool pass=false;
        for (int j=1;j<s;j++) if ((x=mul_mod(x,x,n))==n-1) { pass=true; break
    | ; }
        if (!pass) return false;
    }
    return true;
}
u64 pollard_rho(u64 n) { // n>1인 합성수에 사용한다.
    if (!(n&1)) return 2;
    static mt19937_64 rng(20261014);
    while (true) {
        u64 x=2+rng()%(n-2),y=x,c=1+rng()%(n-1),d=1;
        auto f=[&](u64 a)->u64 { return (u128(mul_mod(a,a,n))+c)%n; };
        while (d==1) { x=f(x); y=f(f(y)); d=gcd(x>y?x-y:y-x,n); }
        if (d!=n) return d;
    }
}
vector<pair<u64,int>> factorize64(u64 n) {
    if (!n) throw invalid_argument("positive integer required");
    vector<u64> st{n}; map<u64,int> factors;
    while (!st.empty()) {
        u64 x=st.back(); st.pop_back(); if (x==1) continue;
        if (is_prime64(x)) ++factors[x];
        else { u64 d=pollard_rho(x); st.push_back(d); st.push_back(x/d); }
    }
    return {factors.begin(),factors.end()};
}
}
```

5.10 가우스 소거법 - RREF, 랭크, 행렬식, 역행렬 [PY/C++]

```
# 가우스 조던 소거로 (기약 행렬, 계수, 행렬식, 역행렬)을 구한다. O(행^2·열).
# 기본은 정확한 Fraction 연산, mod에는 소수를 지정한다. 입력은 보존한다.
# 정의되지 않는 행렬식·역행렬은 None. 정확한 소수 입력은 Fraction으로 전달한다.
from fractions import Fraction

def gauss(matrix, mod=None):
    n = len(matrix)
    m = len(matrix[0]) if n else 0
    if any(len(row) != m for row in matrix):
        raise ValueError('ragged matrix')
    convert = Fraction if mod is None else lambda x: x % mod
    a = [[convert(x) for x in row] for row in matrix]
    inv = [[convert(i == j) for j in range(n)] for i in range(n)]
    norm = (lambda x: x) if mod is None else lambda x: x % mod
    rank, det = 0, convert(1)
    for col in range(m):
        pivot = next((i for i in range(rank, n) if a[i][col]), None)
        if pivot is None:
            continue
        if pivot != rank:
            a[pivot], a[rank] = a[rank], a[pivot]
            inv[pivot], inv[rank] = inv[rank], inv[pivot]
            det = -det
        value = a[rank][col]
        det = norm(det * value)
        scale = 1 / value if mod is None else pow(value, -1, mod)
        a[rank] = [norm(x * scale) for x in a[rank]]
        inv[rank] = [norm(x * scale) for x in inv[rank]]
        for i in range(n):
            if i == rank:
                continue
            scale = a[i][col]
            a[i] = [norm(x - scale * y) for x, y in zip(a[i], a[rank])]
            inv[i] = [norm(x - scale * y) for x, y in zip(inv[i], inv[rank])]
        rank += 1
        if rank == n:
            break
    determinant = norm(det) if rank == n else 0
    return a, rank, determinant if n == m else None, inv if n == m == rank el
    | se None
#pragma once
#include "../common.hpp"
// 가우스 조던 소거: 기약 행렬·계수·행렬식·역행렬. O(행^2·열).
// RealOps는 절대 오차 EPS를 쓰는 근사 연산. Python 기본은 정확한 Fraction.
// 소수 모듈러스의 ModOps는 정확한 유한체 연산.
namespace tn {
struct RealOps {
    using T=long double; T eps=1e-12L;
    T norm(T x) const { return x; }
    bool zero(T x) const { return abs(x)<=eps; }
    T sub(T a,T b) const { return a-b; }
    T mul(T a,T b) const { return a*b; }
    T inv(T x) const { return 1/x; }
    bool better(T a,T b) const { return abs(a)>abs(b); }
};
struct ModOps { // 소수 2<=mod<=2^31-1.
    using T=ll; ll mod;
    T norm(T x) const { x%=mod; return x<0?x+mod:x; }
    bool zero(T x) const { return x==0; }
    T sub(T a,T b) const { return norm(a-b); }
    T mul(T a,T b) const { return i128(a)*b%mod; }
    T inv(T x) const {
        T result=1; for (ll e=mod-2;e;e>>=1,x=mul(x,x)) if (e&1) result=mul(r
    | esult,x);
        return result;
    }
    bool better(T a,T b) const { return b==0 && a!=0; }
};
template<class T> struct GaussResult {
    vector<vector<T>> rref; int rank; optional<T> determinant;
    optional<vector<vector<T>>> inverse;
};
```

```
template<class Ops> GaussResult<typename Ops::T> gauss(
    vector<vector<typename Ops::T>> a,Ops op) {
    using T=typename Ops::T; int n=a.size(),m=n?a[0].size():0,rank=0;
    vector<vector<T>> inv(n,vector<T>(n)); T det=1;
    for (int i=0;i<n;i++) {
        assert(int(a[i].size())==m); inv[i][i]=1;
        for (T& x:a[i]) x=op.norm(x);
    }
    for (int col=0;col<m && rank<n;col++) {
        int p=rank;
        for (int i=rank+1;i<n;i++) if (op.better(a[i][col],a[p][col])) p=i;
        if (op.zero(a[p][col])) continue;
        if (p!=rank) { swap(a[p],a[rank]); swap(inv[p],inv[rank]); det=op.sub
    | (0,det); }
        T pivot=a[rank][col],scale=op.inv(pivot); det=op.mul(det,pivot);
        for (T& x:a[rank]) x=op.mul(x,scale);
        for (T& x:inv[rank]) x=op.mul(x,scale);
        for (int i=0;i<n;i++) if (i!=rank) {
            T factor=a[i][col];
            for (int j=0;j<m;j++) a[i][j]=op.sub(a[i][j],op.mul(factor,a[rank
    | ][j]));
            for (int j=0;j<n;j++) inv[i][j]=op.sub(inv[i][j],op.mul(factor,in
    | v[rank][j]));
        }
        ++rank;
    }
    GaussResult<T> out{a,rank,nullopt,nullopt};
    if (n==m) { out.determinant=rank==n?det:T(0); if (rank==n) out.inverse=in
    | v; }
    return out;
}
}
```

5.11 다항식 곱셈(FFT) [C++]

```
#pragma once
#include "../common.hpp"
// 다항식·비트 합성곱 O(N log N), 공간 O(N).
// NTT/CRT는 정확, FFT는 근사. 뒤쪽 0을 포함한 모든 계수를 반환한다.
namespace tn {
ll mod_power(ll a,ll b,ll mod) {
    ll out=1; for (;b;b>>=1,a=i128(a)*a%mod) if (b&1) out=i128(out)*a%mod;
    return out;
}
void ntt(vector<ll>& a,bool inverse,ll mod=998244353,ll root=3) {
    int n=a.size(); assert(n && !(n&(n-1)) && (mod-1)%n==0);
    for (int i=1,j=0;i<n;i++) {
        int bit=n/2; for (;j&bit;bit>>=1) j^=bit; j^=bit;
        if (i<j) swap(a[i],a[j]);
    }
    for (int len=2;len<=n;len*=2) {
        ll step=mod_power(root,(mod-1)/len,mod);
        if (inverse) step=mod_power(step,mod-2,mod);
        for (int start=0;start<n;start+=len) {
            ll w=1;
            for (int j=start;j<start+len/2;j++) {
                ll u=a[j],v=i128(a[j+len/2])*w%mod;
                a[j]=(u+v)%mod; a[j+len/2]=(u-v+mod)%mod; w=i128(w)*step%mod;
            }
        }
    }
    if (inverse) { ll scale=mod_power(n,mod-2,mod); for (ll& x:a) x=i128(x)*s
    | cale%mod; }
}
vector<ll> ntt_multiply(vector<ll> a,vector<ll> b,ll mod=998244353,ll root=3)
    |  {
    if (a.empty() || b.empty()) return {};
    int size=a.size()+b.size()-1,n=1; while (n<size) n*=2;
    for (ll& x:a) x=(x%mod+mod)%mod;
    for (ll& x:b) x=(x%mod+mod)%mod;
    a.resize(n); b.resize(n); ntt(a,false,mod,root); ntt(b,false,mod,root);
    for (int i=0;i<n;i++) a[i]=i128(a[i])*b[i]%mod;
    ntt(a,true,mod,root); a.resize(size); return a;
}
vector<ll> multiply_mod(vector<ll> a,vector<ll> b,ll mod=998244353) {
    // 1<=mod<=2^31-1, N<=2^24. CRT 복원 범위를 넘으면 예외.
    assert(1<=mod && mod<=INT_MAX); if (a.empty() || b.empty()) return {};
    int size=a.size()+b.size()-1,n=1; while (n<size) n*=2;
    if (n>(1<<24)) throw length_error("NTT length exceeds 2^24");
    if (mod==998244353 && n<=(1<<23)) return ntt_multiply(a,b);
    for (ll& x:a) x=(x%mod+mod)%mod;
    for (ll& x:b) x=(x%mod+mod)%mod;
    ll p=167772161,q=469762049,r=1224736769;
    i128 bound=i128(min(a.size(),b.size()))*(*max_element(a.begin(),a.end()))
        *(*max_element(b.begin(),b.end()));
    if (bound>=i128(p)*q*r) throw overflow_error("CRT bound exceeded");
    auto x=ntt_multiply(a,b,p),y=ntt_multiply(a,b,q),z=ntt_multiply(a,b,r);
    ll ip=mod_power(p,q-2,q),ipq=mod_power(ll(i128(p)*q%r),r-2,r);
    for (int i=0;i<size;i++) {
        i128 value=x[i]+i128((y[i]-x[i]+q)%q)*ip%q*p;
        ll t=ll((z[i]-value%r+r)%r); value+=i128(t)*ipq%r*p*q;
        x[i]=value%mod;
    }
    return x;
}
void fft(vector<complex<long double>>& a,bool inverse=false) {
    int n=a.size(); assert(n && !(n&(n-1)));
    for (int i=1,j=0;i<n;i++) {
        int bit=n/2; for (;j&bit;bit>>=1) j^=bit; j^=bit;
        if (i<j) swap(a[i],a[j]);
    }
    for (int len=2;len<=n;len*=2) {
        auto step=polar(1.0L,(inverse?-2:2)*acosl(-1)/len);
```

```
        for (int s=0;s<n;s+=len) {
            complex<long double> w=1;
            for (int j=s;j<s+len/2;j++) {
                auto u=a[j],v=a[j+len/2]*w;
                a[j]=u+v; a[j+len/2]=u-v; w*=step;
            }
        }
    }
    if (inverse) for (auto& x:a) x/=n;
}
vector<ll> multiply_fft(const vector<ll>& a,const vector<ll>& b) {
    if (a.empty() || b.empty()) return {};
    int size=a.size()+b.size()-1,n=1; while (n<size) n*=2;
    vector<complex<long double>> x(a.begin(),a.end()),y(b.begin(),b.end());
    x.resize(n); y.resize(n); fft(x); fft(y);
    for (int i=0;i<n;i++) x[i]*=y[i]; fft(x,true);
    vector<ll> result(size); for (int i=0;i<size;i++) result[i]=llroundl(x[i]
    | .real());
    return result; // 큰 계수는 반올림 오차 가능. 결과는 ll 범위여야 한다.
}
vector<ll> bitwise_convolution(vector<ll> a,vector<ll> b,char kind) {
    int n=a.size(); assert(n && !(n&(n-1)) && int(b.size())==n);
    assert(kind=='^' || kind=='|' || kind=='&');
    auto transform=[&](vector<ll>& v,bool inverse) {
        for (int step=1;step<n;step*=2) for (int s=0;s<n;s+=2*step)
            for (int j=s;j<s+step;j++) {
                ll x=v[j],y=v[j+step];
                if (kind=='^') v[j]=x+y,v[j+step]=x-y;
                else if (kind=='|') v[j+step]=inverse?y-x:y+x;
                else v[j]=inverse?x-y:x+y;
            }
        if (inverse && kind=='^') for (ll& x:v) x/=n;
    };
    transform(a,false); transform(b,false);
    for (int i=0;i<n;i++) a[i]*=b[i]; transform(a,true); return a;
}
}
```

6. 문자열

6.1 문자열 해싱 [C++]

```
#pragma once
#include "../common.hpp"
// 구간 [l,r) 해시. 전처리 O(N), 질의 O(1).
// 바이트 문자열. 충돌 가능: 실제 값 확인 또는 두 모듈러스 사용.
namespace tn {
struct Hashing {
    ll mod; vector<ll> h{0},p{1};
    Hashing(const string& s,ll base=917,ll mod=998244353):mod(mod) {
        for (unsigned char c:s) h.push_back((h.back()*base+c)%mod),p.push_bac
    | k(p.back()*base%mod);
    }
    ll get(int l,int r) const {
        assert(0<=l && l<=r && r<int(h.size()));
        return (h[r]-h[l]*p[r-l]%mod+mod)%mod;
    }
};
}
```

6.2 문자열 매칭 - KMP [PY/C++]

```
# 접두사 실패 함수를 이용한 패턴 검색 O(N+M). 일치 시작 위치를 반환한다.
# 빈 패턴은 모든 N+1개 경계에 일치한다.

def prefix_function(pattern):
    fail = [0] * len(pattern)
    for i in range(1, len(pattern)):
        j = fail[i - 1]
        while j and pattern[i] != pattern[j]:
            j = fail[j - 1]
        if pattern[i] == pattern[j]:
            j += 1
        fail[i] = j
    return fail

def kmp(text, pattern):
    if not pattern:
        return list(range(len(text) + 1))
    fail, result, j = prefix_function(pattern), [], 0
    for i, char in enumerate(text):
        while j and char != pattern[j]:
            j = fail[j - 1]
        if char == pattern[j]:
            j += 1
        if j == len(pattern):
            result.append(i - j + 1)
            j = fail[j - 1]
    return result
#pragma once
#include "../common.hpp"
// 바이트 패턴 검색 O(N+M). 일치 시작 위치를 반환한다.
namespace tn {
vector<int> prefix_function(const string& p) {
    vector<int> fail(p.size());
    for (int i=1,j=0;i<int(p.size());i++) {
        while (j && p[i]!=p[j]) j=fail[j-1];
        if (p[i]==p[j]) ++j; fail[i]=j;
    }
    return fail;
}
vector<int> kmp(const string& s,const string& p) {
    vector<int> result;
    if (p.empty()) { result.resize(s.size()+1); iota(result.begin(),result.en
    | d(),0); return result; }
```

```
    auto fail=prefix_function(p);
    for (int i=0,j=0;i<int(s.size());i++) {
        while (j && s[i]!=p[j]) j=fail[j-1];
        if (s[i]==p[j]) ++j;
        if (j==int(p.size())) result.push_back(i-j+1),j=fail[j-1];
    }
    return result; // 빈 패턴은 모든 N+1개 경계에 일치한다.
}
}
```

6.3 가장 긴 팰린드롬 부분 문자열 - Manacher [PY/C++]

```
# 구분자를 넣은 2N+1개 중심의 팰린드롬 반지름. O(N).
# 반지름은 원문 팰린드롬 길이이며 구분자는 입력 문자와 충돌하지 않는다.
```

```
def manacher(text):
    separator = object()
    values = [separator]
    for char in text:
        values.extend((char, separator))
    radius = [0] * len(values)
    center = right = 0
    for i in range(len(values)):
        if i < right:
            radius[i] = min(right - i, radius[2 * center - i])
        while (i - radius[i] > 0 and i + radius[i] + 1 < len(values)
               and values[i - radius[i] - 1] == values[i + radius[i] + 1]):
            radius[i] += 1
        if i + radius[i] > right:
            center, right = i, i + radius[i]
    return radius
```

```
#pragma once
#include "../common.hpp"
// 바이트 문자열의 팰린드롬 반지름 O(N).
// 2N+1개 반지름은 원문 팰린드롬 길이. 구분자는 입력과 충돌하지 않는다.
namespace tn {
vector<int> manacher(const string& s) {
    vector<int> v{-1}; for (unsigned char c:s) v.push_back(c),v.push_back(-1)
    | ;
    int n=v.size(); vector<int> radius(n);
    for (int i=0,center=0,right=0;i<n;i++) {
        if (i<right) radius[i]=min(right-i,radius[2*center-i]);
        while (i-radius[i]>0 && i+radius[i]+1<n && v[i-radius[i]-1]==v[i+radi
    | us[i]+1]) ++radius[i];
        if (i+radius[i]>right) center=i,right=i+radius[i];
    }
    return radius;
}
}
```

6.4 문자열 매칭 - Z [PY/C++]

```
# 각 위치에서 전체 문자열의 접두사와 일치하는 길이. O(N), z[0]=N.
```

```
def z_function(text):
    n = len(text)
    z = [0] * n
    if n:
        z[0] = n
    l = r = 0
    for i in range(1, n):
        if i < r:
            z[i] = min(r - i, z[i - l])
        while i + z[i] < n and text[z[i]] == text[i + z[i]]:
            z[i] += 1
        if i + z[i] > r:
            l, r = i, i + z[i]
    return z
```

```
#pragma once
#include "../common.hpp"
// 각 바이트 위치에서 접두사와 일치하는 길이. O(N), z[0]=N.
namespace tn {
vector<int> z_function(const string& s) {
    int n=s.size(); vector<int> z(n); if (n) z[0]=n;
    for (int i=1,l=0,r=0;i<n;i++) {
        if (i<r) z[i]=min(r-i,z[i-l]);
        while (i+z[i]<n && s[z[i]]==s[i+z[i]]) ++z[i];
        if (i+z[i]>r) l=i,r=i+z[i];
    }
    return z;
}
}
```

6.5 접미사 배열 [C++]

```
#pragma once
#include "../common.hpp"
// 접미사 배열과 인접 공통 접두사 길이. 두 배 확장·계수 정렬 O(N log N).
namespace tn {
pair<vector<int>,vector<int>> suffix_array(const string& s) {
    int n=s.size(); if (!n) return {{},{}};
    vector<int> sa(n),rank(n),second(n); iota(sa.begin(),sa.end(),0);
    for (int i=0;i<n;i++) rank[i]=(unsigned char)s[i]+1;
    auto counting=[&](const vector<int>& ids,const vector<int>& key,int class
    | es) {
        vector<int> count(classes+1),out(n);
        for (int i:ids) ++count[key[i]];
        partial_sum(count.begin(),count.end(),count.begin());
        for (int i=n-1;i>=0;i--) out[--count[key[ids[i]]]]=ids[i];
        return out;
    };
    for (int step=1;;step*=2) {
        for (int i=0;i<n;i++) second[i]=i+step<n?rank[i+step]:0;
        int classes=*max_element(rank.begin(),rank.end());
        sa=counting(counting(sa,second,classes),rank,classes);
        vector<int> next(n); classes=1; next[sa[0]]=classes;
```

```
        for (int i=1;i<n;i++) {
            int a=sa[i-1],b=sa[i];
            if (pair<int,int>{rank[a],second[a]}!=pair<int,int>{rank[b],secon
    | d[b]}) ++classes;
            next[b]=classes;
        }
        rank.swap(next); if (classes==n) break;
    }
    vector<int> lcp(n);
    for (int i=0,len=0;i<n;i++) {
        int pos=rank[i]-1; if (pos==0) { len=0; continue; }
        int j=sa[pos-1]; while (i+len<n && j+len<n && s[i+len]==s[j+len]) ++l
    | en;
        lcp[pos]=len; if (len) --len;
    }
    return {sa,lcp}; // lcp[i]=LCP(sa[i-1],sa[i]), lcp[0]=0. 바이트 인덱스.
}
}
```

7. 계산 기하

7.1 2차원 계산 기하 템플릿 + CCW [PY/C++]

```
# 정수 점 (x,y)의 내적·외적·방향·거리 제곱. O(1).
# 실수 좌표를 쓰면 오차 허용 기준을 별도로 정해야 한다.

def add(a, b):
    return a[0] + b[0], a[1] + b[1]

def sub(a, b):
    return a[0] - b[0], a[1] - b[1]

def dot(a, b):
    return a[0] * b[0] + a[1] * b[1]

def cross(a, b):
    return a[0] * b[1] - a[1] * b[0]

def orient(a, b, c):
    return (b[0] - a[0]) * (c[1] - a[1]) - (b[1] - a[1]) * (c[0] - a[0])

def ccw(a, b, c):
    value = orient(a, b, c)
    return (value > 0) - (value < 0)

def distance_squared(a, b):
    return (a[0] - b[0]) ** 2 + (a[1] - b[1]) ** 2
#pragma once
#include "../common.hpp"
// 정수 좌표의 절댓값은 10^9 이하.
// i128로 정확한 방향 판정 O(1). 거리는 제곱값이다.
namespace tn {
using Point=pair<ll,ll>;
Point add(Point a,Point b) { return {a.first+b.first,a.second+b.second}; }
Point sub(Point a,Point b) { return {a.first-b.first,a.second-b.second}; }
i128 dot(Point a,Point b) { return i128(a.first)*b.first+i128(a.second)*b.sec
    | ond; }
i128 cross(Point a,Point b) { return i128(a.first)*b.second-i128(a.second)*b.
    | first; }
i128 orient(Point a,Point b,Point c) { return cross(sub(b,a),sub(c,a)); }
int sign(i128 x) { return (x>0)-(x<0); }
int ccw(Point a,Point b,Point c) { return sign(orient(a,b,c)); }
i128 distance_squared(Point a,Point b) { return dot(sub(a,b),sub(a,b)); }
bool on_segment(Point a,Point b,Point p) { return orient(a,b,p)==0 && min(a,b
    | )<=p && p<=max(a,b); }
}
```

7.2 360도 각도 정렬 [PY/C++]

```
# 원점 우선, +x축부터 반시계 방향 정렬. 같은 각도는 가까운 점 우선. O(N log N).
from functools import cmp_to_key

def polar_sort(points):
    def half(p):
        return 0 if p[1] > 0 or p[1] == 0 and p[0] >= 0 else 1

    def compare(a, b):
        if a == (0, 0) or b == (0, 0):
            return (b == (0, 0)) - (a == (0, 0))
        if half(a) != half(b):
            return half(a) - half(b)
        cross = a[0] * b[1] - a[1] * b[0]
        if cross:
            return -1 if cross > 0 else 1
        da, db = a[0] ** 2 + a[1] ** 2, b[0] ** 2 + b[1] ** 2
        return (da > db) - (da < db)

    return sorted(points, key=cmp_to_key(compare))
#pragma once
#include "Template.hpp"
// 원점 우선, +x축부터 반시계 방향 정렬. 같은 각도는 가까운 점 우선. O(N log N).
namespace tn {
vector<Point> polar_sort(vector<Point> points) {
    auto half=[](Point p) { return !(p.second>0 || (p.second==0 && p.first>=0
    | )); };
    sort(points.begin(),points.end(),[&](Point a,Point b) {
        if (a==Point{0,0} || b==Point{0,0}) return a==Point{0,0} && b!=Point{
    | 0,0};
        if (half(a)!=half(b)) return half(a)<half(b);
        i128 c=cross(a,b); return c?c>0:dot(a,a)<dot(b,b);
    });
    return points; // 각도가 같으면 가까운 점 우선.
}
}
```

7.3 다각형 넓이 [PY]

```
# 경계 순서로 주어진 단순 다각형의 넓이 두 배를 정수로 구한다. O(N).
```

```
def polygon_area2(points):
    return abs(sum(points[i][0] * points[(i + 1) % len(points)][1]
                   - points[i][1] * points[(i + 1) % len(points)][0]
                   for i in range(len(points))))
```

7.4 선분 교차 판정 [PY/C++]

```
# 선분 교차 O(1). (0,None): 없음, (1,점): 한 점, (-1,None): 구간 겹침.
# 교점은 Fraction으로 정확히 계산하며 길이 0인 선분도 처리한다.
from fractions import Fraction

def segment_intersection(a, b, c, d):
    def cross(u, v):
        return u[0] * v[1] - u[1] * v[0]
    def sub(u, v):
        return u[0] - v[0], u[1] - v[1]
    u, v, w = sub(b, a), sub(d, c), sub(c, a)
    det = cross(u, v)
    if det:
        t, s = Fraction(cross(w, v), det), Fraction(cross(w, u), det)
        if 0 <= t <= 1 and 0 <= s <= 1:
            return 1, (a[0] + t * u[0], a[1] + t * u[1])
        return 0, None
    if cross(u, w) or cross(v, w):
        return 0, None
    lo, hi = max(min(a, b), min(c, d)), min(max(a, b), max(c, d))
    if lo > hi:
        return 0, None
    return (1, lo) if lo == hi else (-1, None)
```

```
#pragma once
#include "Template.hpp"
// 점 선분까지 처리하는 선분 교차 판정 O(1).
namespace tn {
struct Intersection {
    int kind; i128 xnum=0,ynum=0,den=1; // 0: 없음, -1: 구간 겹침, 1: 유일한 교
    | 점(정수 비율).
};
Intersection segment_intersection(Point a,Point b,Point c,Point d) {
    Point u=sub(b,a),v=sub(d,c),w=sub(c,a); i128 det=cross(u,v);
    if (det) {
        i128 t=cross(w,v),s=cross(w,u);
        if (det<0) det=-det,t=-t,s=-s;
        if (t<0 || t>det || s<0 || s>det) return {0};
        return {1,i128(a.first)*det+t*u.first,i128(a.second)*det+t*u.second,d
    | et};
    }
    if (cross(u,w) || cross(v,w)) return {0};
    Point lo=max(min(a,b),min(c,d)),hi=min(max(a,b),max(c,d));
    if (lo>hi) return {0};
    if (lo==hi) return {1,lo.first,lo.second,1};
    return {-1};
}
```

7.5 다각형 내부 판별 [PY/C++]

```
# 단순 다각형 내부 판별 O(N). 감김 수를 사용하며 경계를 포함한다.

def point_in_polygon(points, p):
    winding = 0
    for i, a in enumerate(points):
        b = points[(i + 1) % len(points)]
        cross = (b[0] - a[0]) * (p[1] - a[1]) - (b[1] - a[1]) * (p[0] - a[0])
        if cross == 0 and min(a, b) <= p <= max(a, b):
            return True
        if a[1] <= p[1] < b[1] and cross > 0:
            winding += 1
        if b[1] <= p[1] < a[1] and cross < 0:
            winding -= 1
    return winding != 0
```

```
#pragma once
#include "Template.hpp"
// 단순 다각형 내부 판별 O(N).
namespace tn {
bool point_in_polygon(const vector<Point>& points,Point p) {
    int n=points.size(),winding=0;
    for (int i=0;i<n;i++) {
        Point a=points[i],b=points[(i+1)%n]; i128 c=orient(a,b,p);
        if (on_segment(a,b,p)) return true;
        if (a.second<=p.second && p.second<b.second && c>0) ++winding;
        if (b.second<=p.second && p.second<a.second && c<0) --winding;
    }
    return winding!=0; // 경계도 내부로 판정한다.
}
}
```

7.6 볼록 껍질 - Monotone Chain [C++]

```
#pragma once
#include "Template.hpp"
// 단조 체인으로 볼록 껍질을 구한다. O(N log N).
namespace tn {
vector<Point> convex_hull(vector<Point> p) {
    sort(p.begin(),p.end()); p.erase(unique(p.begin(),p.end()),p.end());
    if (p.size()<2) return p;
    auto chain=[](const vector<Point>& points) {
        vector<Point> out;
        for (Point x:points) {
            while (out.size()>1 && orient(out[out.size()-2],out.back(),x)<=0)
    |  out.pop_back();
            out.push_back(x);
        }
        return out;
    };
    auto a=chain(p); reverse(p.begin(),p.end()); auto b=chain(p);
```

```
    a.pop_back(); b.pop_back(); a.insert(a.end(),b.begin(),b.end()); return a
    | ;
} // 반시계 껍질, 끝점 중복 없음. 일직선 입력은 양 끝점만 남긴다.
}
```

7.7 가장 먼 두 점 - Rotating Calipers [C++]

```
#pragma once
#include "Template.hpp"
// 끝점 중복·일직선 내부점 없는 반시계 볼록 껍질의 가장 먼 두 점. O(N).
namespace tn {
optional<pair<Point,Point>> calipers(const vector<Point>& h) {
    int n=h.size(); if (!n) return nullopt;
    if (n<=2) return pair<Point,Point>{h.front(),h.back()};
    auto area=[&](int i,int k,int j) { i128 x=orient(h[i],h[k],h[j]); return
    | x<0?-x:x; };
    i128 best=-1; pair<Point,Point> result; int j=1;
    for (int i=0;i<n;i++) {
        int k=(i+1)%n;
        while (area(i,k,(j+1)%n)>area(i,k,j)) j=(j+1)%n;
        for (int v:{i,k}) { i128 d=distance_squared(h[v],h[j]);
            if (d>best) best=d,result={h[v],h[j]}; }
    }
    return result;
}
}
```

7.8 볼록 다각형 내부 판별 [C++]

```
#pragma once
#include "Template.hpp"
// 반시계 볼록 다각형 내부 판별 O(log N). 끝점 중복·일직선 내부점 제거.
namespace tn {
bool point_in_convex_polygon(const vector<Point>& h,Point p) {
    int n=h.size(); if (n<3) return n && on_segment(h.front(),h.back(),p);
    Point a=h[0]; i128 first=orient(a,h[1],p),last=orient(a,h.back(),p);
    if (first<0 || last>0) return false;
    if (first==0) return on_segment(a,h[1],p);
    if (last==0) return on_segment(a,h.back(),p);
    int l=1,r=n-1;
    while (r-l>1) { int m=(l+r)/2; if (orient(a,h[m],p)>=0) l=m; else r=m; }
    return orient(h[l],h[r],p)>=0;
}
}
```

8. 기타

8.1 가장 긴 증가하는 부분 수열(LIS) [PY/C++]

```
# 엄격히 증가하는 최장 부분 수열 O(N log N).
# 같은 값도 허용하려면 bisect_left를 bisect_right로 바꾼다.
from bisect import bisect_left
def lis(values):
    tails, ends, previous = [], [], [-1] * len(values)
    for i, value in enumerate(values):
        pos = bisect_left(tails, value)
        if pos:
            previous[i] = ends[pos - 1]
        if pos == len(tails):
            tails.append(value)
            ends.append(i)
        else:
            tails[pos], ends[pos] = value, i
    result = []
    i = ends[-1] if ends else -1
    while i != -1:
        result.append(values[i])
        i = previous[i]
    return result[::-1]
```

```
#pragma once
#include "../common.hpp"
// 엄격히 증가하는 최장 부분 수열 O(N log N).
namespace tn {
vector<ll> lis(const vector<ll>& values) {
    vector<ll> tails,result; vector<int> ends,previous(values.size(),-1);
    for (int i=0;i<int(values.size());i++) {
        int p=lower_bound(tails.begin(),tails.end(),values[i])-tails.begin();
        if (p) previous[i]=ends[p-1];
        if (p==int(tails.size())) tails.push_back(values[i]),ends.push_back(i
    | );
        else tails[p]=values[i],ends[p]=i;
    }
    for (int i=ends.empty()?-1:ends.back();i!=-1;i=previous[i]) result.push_b
    | ack(values[i]);
    reverse(result.begin(),result.end()); return result;
} // 같은 값도 허용하려면 upper_bound 사용.
}
```

8.2 이분 탐색 [PY]

```
# 단조 조건의 경계를 구한다. 구간 [lo,hi), 조건 평가 O(log 범위).
def first_true(lo, hi, check):
    # [lo,hi)의 거짓→참 경계. 첫 참의 위치, 없으면 hi.
    while lo < hi:
        mid = (lo + hi) // 2
        if check(mid):
            hi = mid
        else:
            lo = mid + 1
    return lo

def last_true(lo, hi, check):
    # [lo,hi)의 참→거짓 경계. 마지막 참의 위치, 없으면 lo-1.
    return first_true(lo, hi, lambda x: not check(x)) - 1
```

8.3 삼분 탐색 [PY]

```
# 단봉 함수의 최솟값. 닫힌 구간 [lo,hi], 정수는 가장 작은 최소 위치 반환.
# 최소 구간 밖에서 엄격히 감소·증가해야 한다. 정수 O(log 범위).

def ternary_integer(lo, hi, f):
    if lo > hi:
        raise ValueError('empty interval')
    while hi - lo > 3:
        a = lo + (hi - lo) // 3
        b = hi - (hi - lo) // 3
        if f(a) > f(b):
            lo = a + 1
        else:
            hi = b - 1
    return min(range(lo, hi + 1), key=f)

def ternary_real(lo, hi, f, iterations=100):
    for _ in range(iterations):
        a, b = (2 * lo + hi) / 3, (lo + 2 * hi) / 3
        if f(a) > f(b):
            lo = a
        else:
            hi = b
    return (lo + hi) / 2
```

8.4 Python 랜덤, 순서 통계, 비트마스킹 [PY]

```
# 정렬 리스트의 순위·k번째 조회 O(log N)·O(1), 삽입·삭제 O(N).
# 갱신이 많으면 좌표 압축과 펜윅 트리·세그먼트 트리를 사용한다.
from bisect import bisect_left

class OrderedSet:
    def __init__(self, values=()):
        self.values = sorted(set(values))

    def add(self, value):
        i = bisect_left(self.values, value)
        if i == len(self.values) or self.values[i] != value:
            self.values.insert(i, value)

    def discard(self, value):
        i = bisect_left(self.values, value)
        if i < len(self.values) and self.values[i] == value:
            self.values.pop(i)

    def order_of_key(self, value):
        return bisect_left(self.values, value)

    def find_by_order(self, index):
        return self.values[index]

def trailing_zeros(x):
    if x <= 0:
        raise ValueError('positive integer required')
    return (x & -x).bit_length() - 1

def leading_zeros(x, width=64):
    if not 0 <= x < 1 << width:
        raise ValueError('value outside unsigned width')
    return width - x.bit_length()

def next_same_popcount(x):
    # 1인 비트 수가 같은, 현재 값보다 큰 최소 양의 정수.
    if x <= 0:
        raise ValueError('positive integer required')
    low = x & -x
    ripple = x + low
    return ripple | ((ripple ^ x) // low >> 2)

def digit_frequency(n, digit):
    # [1,n]에서 십진 숫자의 등장 횟수. 앞쪽 0 제외, O(log n).
    if not 0 <= digit <= 9:
        raise ValueError('invalid digit')
    answer, place = 0, 1
    while place <= n:
        high, current, low = n // (place * 10), n // place % 10, n % place
        if digit:
            answer += high * place
        elif high:
            answer += (high - 1) * place
        if digit or high:
            if current > digit:
                answer += place
            elif current == digit:
                answer += low + 1
        place *= 10
    return answer
```

8.5 빠른 입력(Fast Input from STDIN) [PY/C++]

```
# 공백으로 구분한 정수 입출력. 전체 읽기는 빠르고, 분할 읽기는 메모리를 절약한다.

import sys

def read_ints(stream=None):
    if stream is None:
        stream = sys.stdin.buffer
    return list(map(int, stream.read().split()))

def iter_ints(stream=None, chunk_size=1 << 16):
    # 입력은 공백으로 구분한 올바른 십진 정수여야 한다.
    if stream is None:
        stream = sys.stdin.buffer
    if chunk_size <= 0:
        raise ValueError('positive chunk size required')
    value, sign, active = 0, 1, False
    while True:
        chunk = stream.read(chunk_size)
        if not chunk:
            break
```

```
        for c in chunk:
            if 48 <= c <= 57:
                value = value * 10 + c - 48
                active = True
            elif c == 45:
                sign = -1
            else:
                if active:
                    yield sign * value
                value, sign, active = 0, 1, False
    if active:
        yield sign * value

def write_lines(values, stream=None):
    if stream is None:
        stream = sys.stdout
    stream.write('\n'.join(map(str, values)))
#pragma once
#include "../common.hpp"
// 빠른 정수 입력. 먼저 동기화를 끈 cin 사용을 고려한다.
// 64KiB 버퍼 정수 입력, LLONG_MIN 포함. 시간 O(입력 크기).
// 공백으로 구분한 ll 범위의 올바른 십진 정수만 입력한다.
namespace tn {
struct FastInput {
    FILE* file; char buffer[1<<16]; size_t pos=0,size=0;
    explicit FastInput(FILE* f=stdin):file(f) {}
    int get() {
        if (pos==size) { size=fread(buffer,1,sizeof buffer,file); pos=0; if (
    | !size) return EOF; }
        return (unsigned char)buffer[pos++];
    }
    bool read(ll& value) {
        int c; do { c=get(); } while (c!=EOF && c<=32);
        if (c==EOF) return false;
        bool negative=c=='-'; if (c=='-' || c=='+') c=get();
        u64 x=0; while ('0'<=c && c<='9') x=x*10+c-'0',c=get();
        value=negative?(x==(1ULL<<63)?LLONG_MIN:-ll(x)):ll(x); return true;
    }
};
// 표준 입력은 동기화를 끈 cin으로 대체할 수 있다.
// 같은 입력에서 FastInput/fread와 비동기 cin을 혼용하지 않는다.
}
```

8.6 구간별 약수 최대 개수, 최대 소수

원본의 수치표를 보존했습니다. 각 행의 지수는 오른쪽 소수 목록에 대응합니다.

< 10^k	number	divisors	2 3 5 71113171923293137
1	6	4	1 1
2	60	12	2 1 1
3	840	32	3 1 1 1
4	7560	64	3 3 1 1
5	83160	128	3 3 1 1 1
6	720720	240	4 2 1 1 1 1
7	8648640	448	6 3 1 1 1 1
8	73513440	768	5 3 1 1 1 1 1
9	735134400	1344	6 3 2 1 1 1 1
10	6983776800	2304	5 3 2 1 1 1 1 1
11	97772875200	4032	6 3 2 2 1 1 1 1
12	963761198400	6720	6 4 2 1 1 1 1 1 1
13	9316358251200	10752	6 3 2 1 1 1 1 1 1 1
14	97821761637600	17280	5 4 2 2 1 1 1 1 1 1
15	866421317361600	26880	6 4 2 1 1 1 1 1 1 1 1
16	8086598962041600	41472	8 3 2 2 1 1 1 1 1 1 1
17	74801040398884800	64512	6 3 2 2 1 1 1 1 1 1 1 1
18	897612484786617600	103680	8 4 2 2 1 1 1 1 1 1 1 1

< 10^k	prime	# of prime	< 10^k	prime
1	7	4	10	9999999967
2	97	25	11	99999999977
3	997	168	12	999999999989
4	9973	1229	13	9999999999971
5	99991	9592	14	99999999999973
6	999983	78498	15	999999999999989
7	9999991	664579	16	9999999999999937
8	99999989	5761455	17	99999999999999997
9	999999937	50847534	18	999999999999999989

8.7 카탈란 수, 심슨 적분, 그런디 정리, 픽의 정리, 페르마 포인트, 오일러 정리

```
# 바닥 함수 합·카탈란 수·수치 적분·근 찾기·집합 분할 수.
from math import comb

def floor_sum(n, m, a, b):
    if n < 0 or m <= 0:
        raise ValueError('n >= 0 and m > 0 required')
    answer = 0
    while True:
        q, a = divmod(a, m)
        answer += q * n * (n - 1) // 2
        q, b = divmod(b, m)
        answer += q * n
        top = a * n + b
        if top < m:
            return answer
        n, b = divmod(top, m)
        m, a = a, m
```

```
def catalan(n):
    return comb(2 * n, n) // (n + 1)

def simpson(f, a, b, steps=1000):
    if steps <= 0 or steps % 2:
        raise ValueError('positive even steps required')
    h = (b - a) / steps
    return h / 3 * (f(a) + f(b) + sum((4 if i % 2 else 2) * f(a + i * h)
                                      for i in range(1, steps)))

def newton(f, derivative, x, tolerance=1e-12, iterations=100):
    # 뉴턴법으로 근을 찾는다. 수렴 실패 가능: 결과의 잔차를 확인한다.
    for _ in range(iterations):
        value = f(x)
        if abs(value) <= tolerance:
            return x
        slope = derivative(x)
        if slope == 0:
            raise ValueError('zero derivative')
        x -= value / slope
    if abs(f(x)) <= tolerance:
        return x
    raise ValueError('iteration limit reached')

def stirling2_row(n, k):
    row = [1] + [0] * k
    for _ in range(n):
        for j in range(k, 0, -1):
            row[j] = j * row[j] + row[j - 1]
        row[0] = 0
    return row

def bell(n):
    return sum(stirling2_row(n, n))

def derangements(n):
    a, b = 1, 0
    if n == 0:
        return a
    for i in range(2, n + 1):
        a, b = b, (i - 1) * (a + b)
    return b
```

카탈란 수: C(n) = comb(2*n,n) // (n+1). 처음 값은 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, 208012, 742900. 길이 2n의 올바른 괄호 문자열, 잎 n+1개의 순서 있는 full binary tree, 볼록 (n+2)각형의 삼각 분할을 센다.

심슨 공식: n은 양의 짝수, h=(b-a)/n, x_i=a+i*h. 적분 근삿값은 h/3 * (f(x_0)+f(x_n)+4*sum(f(x_i), i 홀수) +2*sum(f(x_i), 0<i<n 짝수)). f의 4계 도함수가 연속이고 절댓값이 M 이하이면 절대 오차는 M*(b-a)*h^4/180 이하.

Nim: 일반 규칙에서는 모든 더미 크기의 XOR가 0이면 후공 승리, 아니면 선공 승리. Grundy 값은 가능한 다음 상태의 값들의 mex(포함되지 않은 최소 비음수 정수). 독립적인 게임의 합은 각 Grundy 값의 XOR. 유한 비순환 impartial normal-play 게임에 적용한다.

Subtraction game: 한 번에 한 더미에서 1..k개를 제거할 때 각 더미 크기를 k+1로 나눈 나머지가 Grundy 값이다. 이 값들의 XOR로 판단한다.

Index-k Nim (Moore's Nim): 한 번에 1..k개 더미를 고르고 각 더미에서 양의 개수를 제거한다. 각 이진 자릿수에서 1의 개수를 k+1로 나눈 나머지가 모두 0인 상태가 패배 상태다. 이 나머지 벡터를 일반적인 Grundy 값으로 취급하지 않는다.

Misere Nim: 마지막 돌을 가져가는 쪽이 패배한다. 0인 더미를 제외하고 모두 크기 1이면 양의 더미 수가 홀수일 때 후공 승리. 크기 2 이상 더미가 있으면 XOR가 0일 때 후공 승리. 빈 시작 상태는 게임 규칙의 종료 정의를 확인한다.

Pick 정리: 단순 격자 다각형에서 넓이 A = 내부 격자점 수 I + 경계 격자점 수 B/2 - 1. 경계 격자점 수는 각 변의 gcd(abs(dx), abs(dy)) 합이다. floor_sum(n,m,a,b)는 i=0..n-1에 대해 floor((a*i+b)/m)를 합한다. 아래 Reference.py에 반복형 구현을 포함했다.

페르마 포인트: 삼각형 꼭짓점까지 거리 합을 최소화한다. 120도 이상인 각이 있으면 그 꼭짓점이다. 모든 각이 120도 미만이면 각 변 바깥쪽 정삼각형의 새 꼭짓점과 반대 꼭짓점을 잇는 선분들이 만나는 점이다. 최소 거리 합은 sqrt((a*a+b*b+c*c+4*sqrt(3)*A)/2), A는 삼각형 넓이.

오일러 정리: gcd(a,m)=1이면 a^phi(m) = 1 (mod m). 이 경우 지수를 phi(m)로 나눠 줄일 수 있다. 서로소가 아닐 때 무조건 지수를 줄이면 틀린다. 충분한 지수 e >= ceil(log2(m)), m>=2에서는 a^e = a^(e % phi(m) + phi(m)) (mod m)를 사용할 수 있다. 작은 지수는 원래대로 계산한다.

소수 p에 대해 g^0+...+g^(p-2)는 g=1 (mod p)이면 -1 (mod p), g가 0도 1도 아니면 0 (mod p)이다. g=0이면 합은 1이므로 별도 처리한다.

8.8 경우의 수 - 포함 배제, 스털링 수, 벨 수

구별되는 n개 공을 구별되는 k개 상자에 모두 비지 않게 넣는 경우(전사 함수): sum((-1)^(k-i)*comb(k,i)*i^n, i=0..k) = k!*S2(n,k). 원문 첫 항목의 '공 구별 X'를 '공 구별 O'로 정정했다. 구별되지 않는 공, 구별되는 양의 상자라면 comb(n-1,k-1)이다.

구별되는 공, 구별되지 않는 k개 비지 않은 상자: 제2종 스털링 수 S2(n,k)=k*S2(n-1,k)+S2(n-1,k-1). S2(0,0)=1, 유효 범위 밖은 0. 포함 배제식은 위 전사 함수 수를 k!로 나눈 값이다. 모듈러 나눗셈에는 역원의 존재 조건을 확인한다.

벨 수: B(n)=sum(S2(n,k), k=0..n). 최대 k개 블록이면 B(n,k)=sum(S2(n,j), j=0..k). B(n)=sum(comb(n-1,i)*B(i), i=0..n-1), B(0)=1. 최대 k개 식은 sum(i^n/i! * sum((-1)^j/j!, j=0..k-i), i=0..k)로도 정리된다. 이 식의 나눗셈은 유리수 또는 유효한 모듈러 역원으로 계산한다.

완전 순열: D(0)=1, D(1)=0, D(n)=(n-1)*(D(n-1)+D(n-2)).

부호 없는 제1종 스털링 수: c(n,k)=(n-1)*c(n-1,k)+c(n-1,k-1). 부호 있는 제1종 스털링 수: s(n,k)=s(n-1,k-1)-(n-1)*s(n-1,k). 둘 다 (0,0)=1. 원문의 signed 점화식 부호를 정정했다.

정수 n을 정확히 k개의 양의 정수로 분할: p(n,k)=p(n-1,k-1)+p(n-k,k), p(0,0)=1. 전체 분할 수: p(0)=1, p(n)=sum((-1)^(k-1)*(p(n-k*(3*k-1)//2)+p(n-k*(3*k+1)//2)), k>=1; 음수 인자의 p는 0. 원문의 부호를 정정했다.

카탈란 등가식: C(n)=comb(2*n,n)-comb(2*n,n+1)=(2*n)!/(n!*(n+1)!). C(0)=1, C(n)=sum(C(i)*C(n-1-i), i=0..n-1) for n>=1.

8.9 삼각형의 오심 - 외심, 내심, 무게중심, 수심, 방심

비퇴화 삼각형에서 변 a,b,c는 각각 꼭짓점 A,B,C의 맞은편 길이. 반둘레 p=(a+b+c)/2. 넓이 K=sqrt(p*(p-a)*(p-b)*(p-c)). 외접 반지름 R=a*b*c/(4*K), 내접 반지름 r=K/p.

중선 m_a=sqrt(2*b*b+2*c*c-a*a)/2. 각 이등분선 길이 t_a=sqrt(b*c*(1-(a/(b+c))^2)). 사인 법칙 sin(A)/a=1/(2*R). 코사인 법칙 a*a=b*b+c*c-2*b*c*cos(A). 탄젠트 법칙 (a+b)/(a-b)=tan((A+B)/2)/tan((A-B)/2), 분모가 0이면 별도 처리.

무게 (alpha,beta,gamma)에 대응하는 점은 (alpha*A+beta*B+gamma*C)/(alpha+beta+gamma). X=b*b+c*c-a*a, Y=a*a+c*c-b*b, Z=a*a+b*b-c*c라 두면 외심의 무게는 (a*a*X,b*b*Y,c*c*Z), 내심은 (a,b,c), 무게중심은 (1,1,1), 수심은 (Y*Z,Z*X,X*Y), A에 대응하는 방심은 (-a,b,c). 직각 삼각형의 수심 공식은 0/0이 될 수 있으므로 직각 꼭짓점을 직접 반환한다.

8.10 미적분, 뉴턴 랩슨법

실수 정의역에서 (arcsin x)'=1/sqrt(1-x*x), (arccos x)'=-1/sqrt(1-x*x), (arctan x)'=1/(1+x*x), (tan x)'=1+tan(x)^2.

적분 tan(a*x) dx = -ln(abs(cos(a*x)))/a + C. 적분 x*sin(a*x) dx = (sin(a*x)-a*x*cos(a*x))/(a*a)+C. 두 식 모두 a != 0.

Newton: x_next=x-f(x)/f'(x). 초기 추정치와 도함수에 따라 발산·순환할 수 있다. 잔차, 도함수 0, 반복 상한을 확인한다. 이분 탐색처럼 수렴이 보장되는 방법과 혼용할 수 있다.

Green 정리: 경계 C에서 integral(L dx + M dy) = 영역 D에서 double_integral(dM/dx - dL/dy) dx dy. C는 양의 방향의 조각별 매끄러운 단순 폐곡선, L,M은 D를 포함하는 열린 영역에서 연속 편도함수를 가져야 한다.

8.11 문제 풀이 체크리스트

비슷한 문제를 풀어본 적이 있는가? 단순한 완전 탐색에서 시작할 수 있는가? 예제를 직접 풀고 그 과정을 수식화할 수 있는가? 문제를 단순화하거나 그림으로 표현할 수 있는가?

문제를 작은 부분으로 분해할 수 있는가? 뒤에서부터 풀 수 있는가? 순서를 강제할 수 있는가? 답을 특정한 정규형으로 제한할 수 있는가?

구간을 통째로 선택하는 문제는 플로우와 구간 자료구조를 고려한다. 원문 힌트 (i,i+1,k,0), (s,e,1,w), (N,T,k,0)는 (시작,끝,용량,비용) 간선 예시다. 모델과 비용 부호는 문제에 맞춰 정한다.

a=b를 만족시켜야 한다면 a만 변경, b만 변경, 둘 다 변경, 역방향 변환을 각각 생각한다. 당연하다고 가정한 내용을 다시 점검한다.

확률 문제는 DP와 이분 탐색을 고려한다(원문 예: NYPC 2019 Finals C). 최대·최소 문제는 이분 탐색, 그리디의 교환 논증·접두사 고정, 순서 고정 DP를 검토한다.

Python 제출 전: 입력량과 메모리, 정수 연산 비용, 0-based/[l,r) 경계, 빈 입력·중복·연결되지 않은 그래프, 음수 가중치, 재귀 사용 여부, float 오차, 해시 충돌을 확인한다. 로컬 테스트 통과와 온라인 저지 통과를 구분한다.

9. Python 추가 참조

9.1 Python 빠른 참조 [PY]

```
import sys, math
from collections import Counter, defaultdict
from itertools import accumulate, combinations, permutations
input = sys.stdin.readline

a = list(map(int, input().split()))  # 정수 배열 입력
x, y = map(int, input().split())  # 언패킹 (map은 iterator)
s = input().strip()  # 양끝 공백 제거 (줄바꿈 포함)
print(*a)  # 공백 구분 출력
print(*a, sep='\n')  # 줄바꿈 출력
print(' '.join(map(str, a)))  # str 변환 후 합치기
for i, x in enumerate(a): ...  # (인덱스, 값)
for x, y in zip(A, B): ...  # 짧은 쪽 길이까지만
q, r = divmod(x, y)  # 몫, 나머지 (y != 0)
sum(x > 0 for x in a)  # 조건 만족 개수
any(x > 0 for x in a)  # 하나라도 True
all(x > 0 for x in a)  # 모두 True (빈 배열은 True)
min(a, default=-1)  # 빈 iterable 허용
max(a, key=lambda x: x[1])  # 두 번째 요소 기준 최대
a.index(x)  # 첫 인덱스, 없으면 ValueError
a.count(x)  # O(N)
a.append(x)  # 끝 삽입: 분할 상환 O(1)
a.insert(i, x)  # 중간 삽입: O(N)
a.pop()  # 끝 삭제: O(1)
a.pop(0)  # 앞 삭제: O(N) -> deque 사용
b = a[::-1]  # 역순 사본 O(N)
a.reverse()  # 제자리 뒤집기, None 반환
b = a[:]  # 얕은 복사 (내부 객체 공유)
b = a  # 복사 X (같은 객체 참조)
grid = [[0] * m for _ in range(n)]  # 2차원 배열: 행 각각 생성
grid = [[0] * m] * n  # 금지! 모든 행이 같은 객체
st = set(a)  # 중복 제거, 순서 보장 X
st.add(x); st.discard(x)  # discard는 없어도 오류 X
st.remove(x)  # 없으면 KeyError
x in st; x in dictionary  # 평균 O(1)
x in a  # 리스트 탐색 O(N)
Counter(a)[x]  # 등장 횟수
defaultdict(int)['missing']  # 기본값 0
list(accumulate(a))  # 누적합
list(combinations(a, 2))  # 조합 (생성량 주의)
list(permutations(a, 2))  # 순열 (생성량 주의)
ord('a'); chr(97)  # 문자 <-> 코드포인트
math.isqrt(n)  # floor(sqrt(n)), 정수 정확
math.ceil(x); math.floor(x)  # 올림 / 내림
len(str(abs(n)))  # 자릿수 (n=0이면 1)
list(map(int, str(n)))  # 양의 정수 자릿수 추출
```

9.2 문법과 성능 주의사항 [PY]

```
# sort·reverse·set.add의 반환값은 None: 결과를 원래 변수에 대입하지 않는다.
# a[:k]는 시간·공간 O(k). 루프 안에서 반복 복사하지 않는다.
# 문자열 += 반복 대신 조각을 리스트에 모아 join한다.
# max·min·sum·list·str 등 내장 이름을 변수로 덮어쓰지 않는다.
# 최댓값 우선, 행·열 최솟값 우선: max((v,-r,-c) for v,r,c in arr).
# 이 방식의 반환 좌표는 음수이므로 다시 부호를 뒤집어 복원한다.
# 복잡도 감각: 2^N은 N~20, N^2은 N~10^3, N log N은 N~10^5.
# 위 수치는 통과 보장이 아니다. 연산 비용·시간 제한·메모리로 판단한다.
```

9.3 곱셈표의 k번째 수 [PY]

```
from python.Misc.BinarySearch import first_true

def kth_multiplication_table(n, k):
    # n*n 곱셈표의 k번째 값. 1<=k<=n*n, O(n log n).
    def count_leq(x):
        return sum(min(n, x // i) for i in range(1, n + 1))
    # 개수가 k와 같아도 중단하지 않고, k 이상인 첫 위치를 찾는다.
    return first_true(1, n * n + 1, lambda x: count_leq(x) >= k)
```

9.4 편집 거리와 DP 점검 [PY]

```
def edit_distance(a, b):
    # 삽입·삭제·교체 비용 각 1. 시간·공간 O(len(a)*len(b)).
    n, m = len(a), len(b)
    dp = [[0] * (m + 1) for _ in range(n + 1)]
    for i in range(n + 1):
        dp[i][0] = i  # 빈 문자열로 만드는 삭제 비용
    for j in range(m + 1):
        dp[0][j] = j  # 빈 문자열에서 시작하는 삽입 비용
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            dp[i][j] = min(dp[i-1][j] + 1,
                           dp[i][j-1] + 1,
                           dp[i-1][j-1] + (a[i-1] != b[j-1]))
    return dp[n][m]
# DP 점검: 상태 의미 → 초기값 → 점화식 → 순회 순서 → 답 위치.
# 최댓값의 불가능 상태는 -INF, 경우의 수는 모듈러 적용 확인.
# 1차원 압축 시 이전 행과 현재 행의 값을 섞지 않는다.
```

9.5 수선의 발·거리·대칭점·교점 [PY]

```
import math
from python.Geometry.Template import sub, dot, cross, distance_squared

def area2(a, b, p):
    # 삼각형 넓이의 두 배. 정수 좌표에서는 정확한 정수.
    return abs(cross(sub(b, a), sub(p, a)))

def perpendicular_foot(a, b, p):
    # 직선 AB 위 수선의 발과 비율 t. A!=B, 실수 근삿값 반환.
    v = sub(b, a)
    denom = dot(v, v)
    if denom == 0:
        raise ValueError('A와 B가 같아 직선이 정의되지 않음')
    t = dot(sub(p, a), v) / denom
    h = (a[0] + t * v[0], a[1] + t * v[1])
    return h, t  # 0<=t<=1이면 발이 선분 위에 있다.

def point_line_dist(a, b, p):
    # 점과 무한 직선 사이 거리. A!=B, 실수 오차 주의.
    denom = distance_squared(a, b)
    if denom == 0:
        raise ValueError('A와 B가 같아 직선이 정의되지 않음')
    return area2(a, b, p) / math.sqrt(denom)

def reflected_point(a, b, p):
    # 직선 AB에 대한 점 P의 대칭점. A!=B.
    h, _ = perpendicular_foot(a, b, p)
    return 2 * h[0] - p[0], 2 * h[1] - p[1]

def intersection_x(a, b, c):
    # 직선 AB와 x=c의 교점. 평행·일치·점 입력은 None.
    dx = b[0] - a[0]
    if dx == 0:
        return None
    t = (c - a[0]) / dx
    return c, a[1] + t * (b[1] - a[1])

def intersection_y(a, b, c):
    # 직선 AB와 y=c의 교점. 평행·일치·점 입력은 None.
    dy = b[1] - a[1]
    if dy == 0:
        return None
    t = (c - a[1]) / dy
    return a[0] + t * (b[0] - a[0]), c
```

9.6 제출 전 추가 점검 [PY]

```
# 이분 탐색: 개수==k에서 멈추지 말고 첫 참·마지막 참을 찾았는가?
# DP: 첫 행·열, 불가능 상태, 압축 배열 갱신 방향을 확인했는가?
# 기하: 좌표 (x,y)와 배열 (행,열), 평행·일치·점 입력을 구분했는가?
# 성능: 반복문 안의 리스트 탐색·복사·슬라이싱이 누적되지 않는가?
# 출력: 개행·공백·자료형이 요구 형식과 일치하는가?
```